Lec4

Meiying Zhang

COE201 - AI Large Models

- Lectures: Monday 10:20-12:10AM
- Software labs: Monday 4:20-6:10PM
- Office hours: Thursday 2:30-3:30PM
- For personal /sensitive matters: zhangmy@sustech.edu.cn

Recap

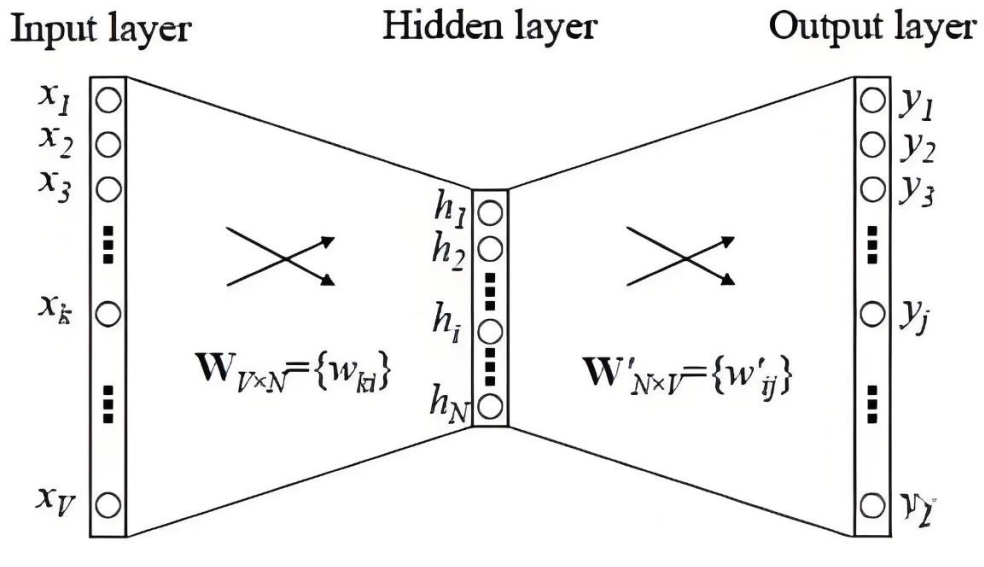
One-hot representation

- Sparse
- Each word is just a string or indices in the vocabulary list
- Hard to compute word relationships

Recap

Distributed Representation—>Word2vec

- Alternative: we want to represent words as short (50-300 dimensional) & dense(real-valued) vectors
    - The basis for modern NLP systems



$$v_{cat} = \begin{pmatrix} -0.224 \\ 0.130 \\ -0.290 \\ \vdots \\ 0.276 \end{pmatrix}_{n\times 1} \qquad v_{dog} = \begin{pmatrix} -0.124 \\ 0.430 \\ -0.200 \\ \vdots \\ 0.329 \end{pmatrix}_{n\times 1}$$

$$v_{the} = \begin{pmatrix} 0.234 \\ 0.266 \\ 0.238 \\ \vdots \\ -0.199 \end{pmatrix}_{n\times 1} \qquad v_{language} = \begin{pmatrix} 0.194 \\ -0.447 \\ 0.780 \\ \vdots \\ 0.953 \end{pmatrix}_{n\times 1}$$

- Each word is represented by a low-dimensional (e.g., d = 300), real-valued vector
- Each coordinate/dimension of the vector doesn't have a particular interpretation

Recap

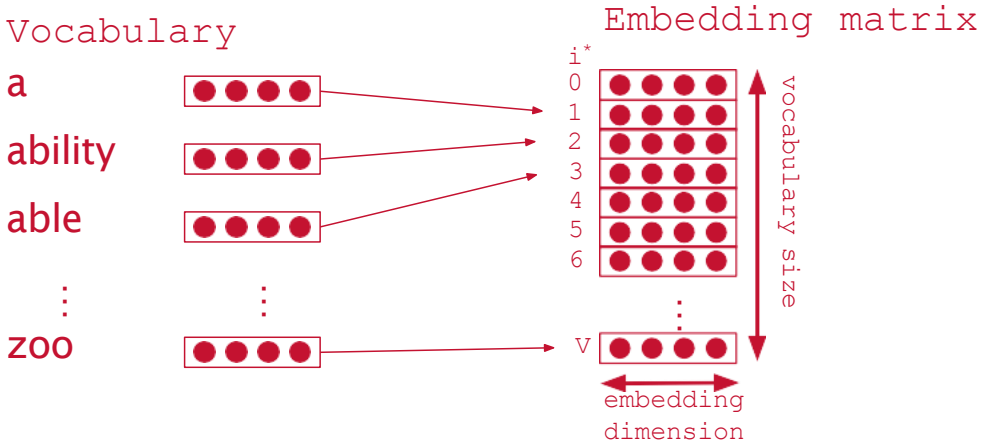
A tokenizer takes text and turns it into a sequence of discrete tokens.

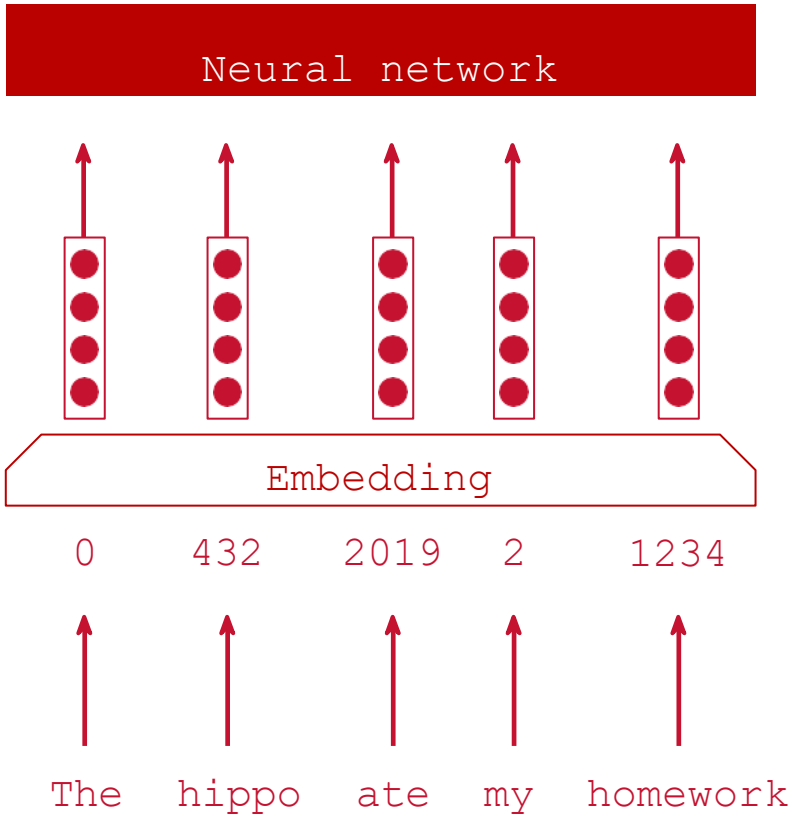
- **Subword tokenization**
  - High-frequency words remain as complete whole words.
  - Low-frequency words are split into meaningful subword units.
  - No out-of-vocabulary (OOV), thus preventing information loss.
  - The ability to learn relationships between prefixes, suffixes, and other morphemes.

Methods	Typical Models
BPE	GPT, LLaMA, BART, ChatGLM-6B
WordPiece	BERT, DistilBERT, MobileBERT
Unigram	AlBERT, T5, mBART, XLNet

Recap

- The LM takes as input the vectors of each token in the input sequence.





Recap

Position Embedding---Sinusoidal Positional Encoding (SPE)

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d}\right)$$

Where pos: position
d: the embedding dimension
i: the ith dimension

- $PE(t+k)$ can be expressed as a linear function of $PE(t)$
- $PE(t+k)PE(t)$ only depends on the relative position k.
- $PE(t+k)PE(t) = PE(t)PE(t-k)$ is symmetric!
- As k increases, $PE(t+k)PE(t)$, there is a long-term decay.

Recap

Position Embedding---Rotary Position Embedding (RoPE)

$$\boxed{e^{im\theta} = \cos(m\theta) + isin(m\theta)}$$

$$f_q(\boldsymbol{x}_m, m) = (\boldsymbol{W}_q \boldsymbol{x}_m)e^{im\theta}$$

$$f_k(\boldsymbol{x}_n, n) = (\boldsymbol{W}_k \boldsymbol{x}_n)e^{in\theta}$$

$$f_q(\boldsymbol{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta) \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} q_m^{(1)} \\ q_m^{(2)} \end{pmatrix}$$

$$f_k(\boldsymbol{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta) \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} k_m^{(1)} \\ k_m^{(2)} \end{pmatrix}$$

The inner product encodes position information only in the relative form:

$$\langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n) \rangle = g(\boldsymbol{x}_m, \boldsymbol{x}_n, m - n).$$

inner product

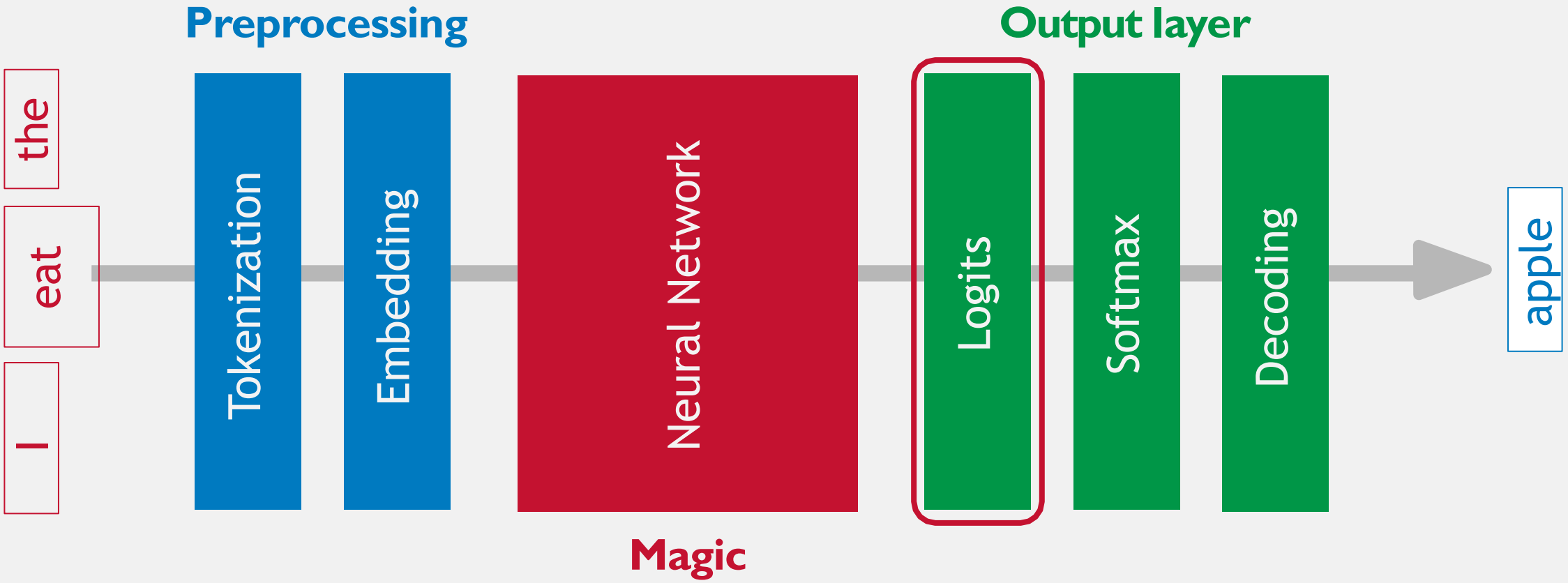
A function $g(\cdot)$, which takes only the token embeddings x_m and x_n , and their relative position $m - n$ as input variables

Agenda

1. Output layers of language model
   - Logits
   - Softmax
   - Decoding/Sampling

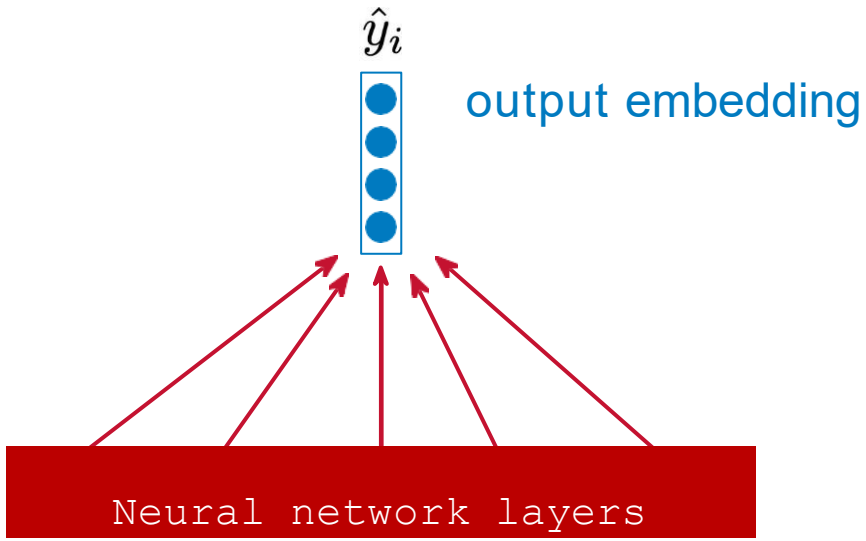
2. Neural Language Modeling
   - RNNs as Language Models
   - Attention Mechanism

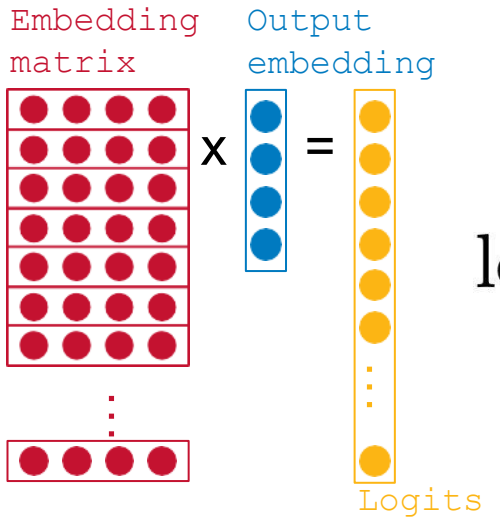
Output layers of Language Models


I
eat
the
Preprocessing
Tokenization
Embedding
Magic
Neural Network
Output layer
Logits
Softmax
Decoding
apple

Logits

- Ideally, the output embedding would be as close as possible to the embedding of the true next token.
- We multiply the predicted embedding by the vocabulary embedding matrix to get a score for each vocabulary word (logits).

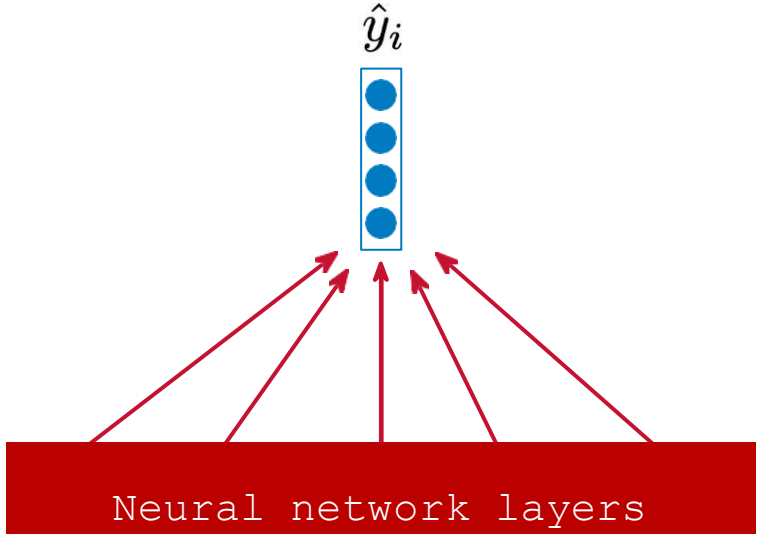


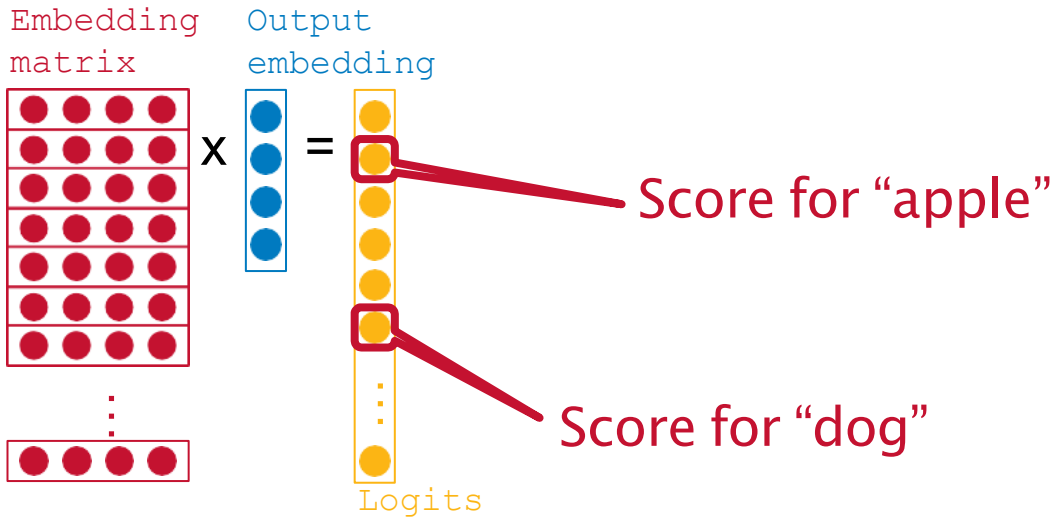


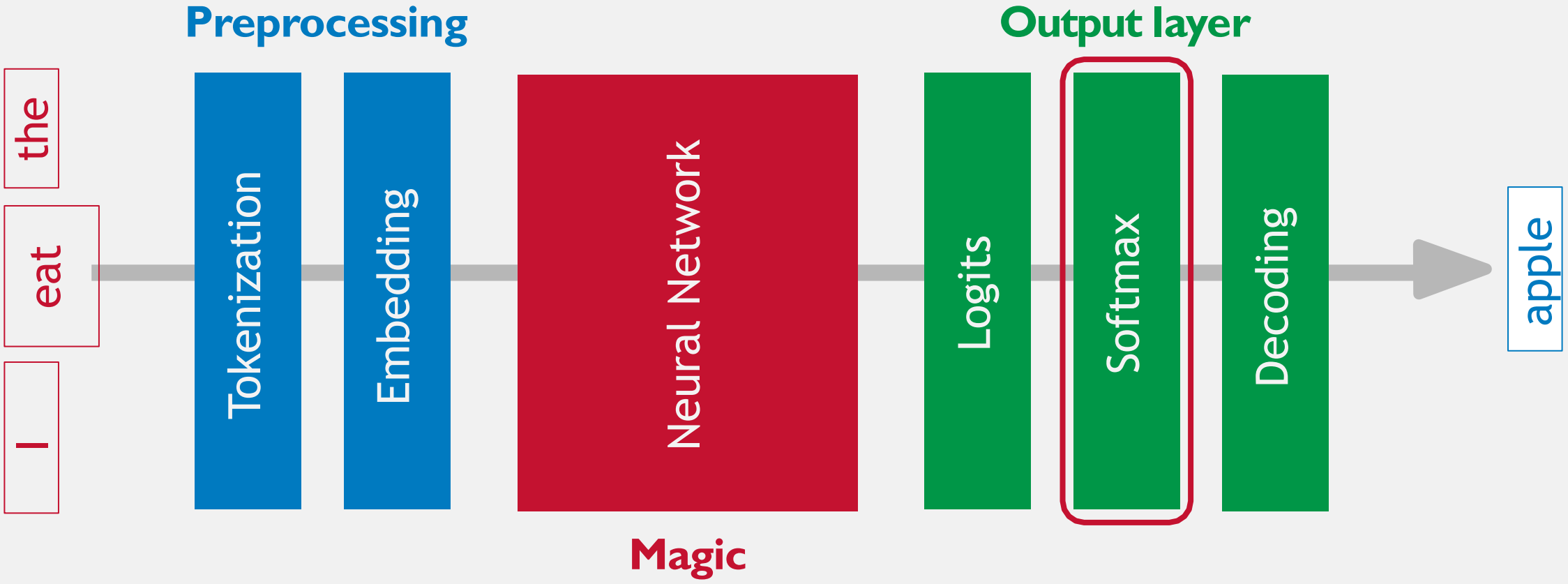
$$\text{logits} = \mathbf{E}\hat{y}_i$$

Logits

- Ideally, the output embedding would be as close as possible to the embedding of the true next token.
- We multiply the predicted embedding by the vocabulary embedding matrix to get a score for each vocabulary word (logits).






Preprocessing
Output layer
the
eat
I
Tokenization
Embedding
Neural Network
Logits
Softmax
Decoding
apple
Magic

Softmax Function

Scores

$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix}$$

Softmax function

$$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Probabilities

$$\begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$

Token Probabilities

We multiply the predicted embedding by our vocabulary embedding matrix to get a score for each vocabulary word. These scores are referred to as logits.

The softmax function is used to turn the logits into probabilities.

$$P(y_i \mid y_{1:i-1}) = \frac{\exp(\mathbf{E}\hat{y}_i[i^*])}{\sum_{j=1}^{|V|} \exp(\mathbf{E}\hat{y}_i[j^*])}$$

Example: Suppose we are trying to predict the 5th word in the sequence “the dog chased the”. We want to know the probability the next word is “cat”.

$$P(y_i = \text{“cat”} \mid [\text{“the”}, \text{“dog”}, \text{“chase”}, \text{“the”}]) = \frac{\exp(\text{logit for “cat”})}{\sum \exp(\text{logit for } t \in V)} = 0.321$$

Negative Log Likelihood

$$\mathcal{L} = -\sum_{t=1}^{T} \log \boxed{P(y_t = i* \mid y_{1:t-1})}$$

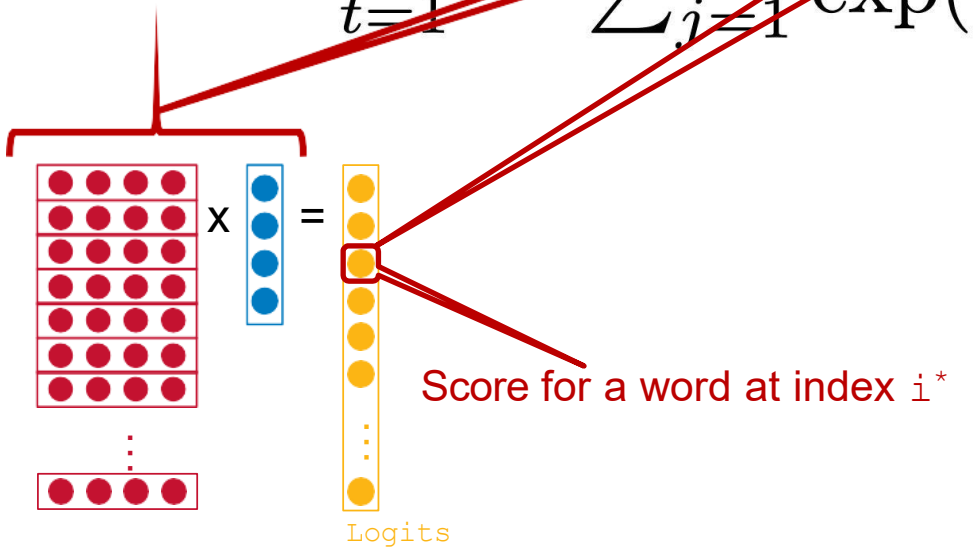
Lec3

The probability the language model assigns to the true t^{th} word in the target sequence.

Negative Log Likelihood

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(y_t = i* \mid y_{1:t-1})$$

$$= -\sum_{t=1}^{T} \log \frac{\exp(\mathbf{E}\hat{y}_t[i^*])}{\sum_{j=1}^{|V|} \exp(\mathbf{E}\hat{y}_t[j^*])}$$



Recall:

$$P(y_i \mid y_{1:i-1}) = \frac{\exp(\mathbf{E}\hat{y}_i[i^*])}{\sum_{j=1}^{|V|} \exp(\mathbf{E}\hat{y}_i[j^*])}$$

Negative Log Likelihood

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(y_t = i* \mid y_{1:t-1})$$

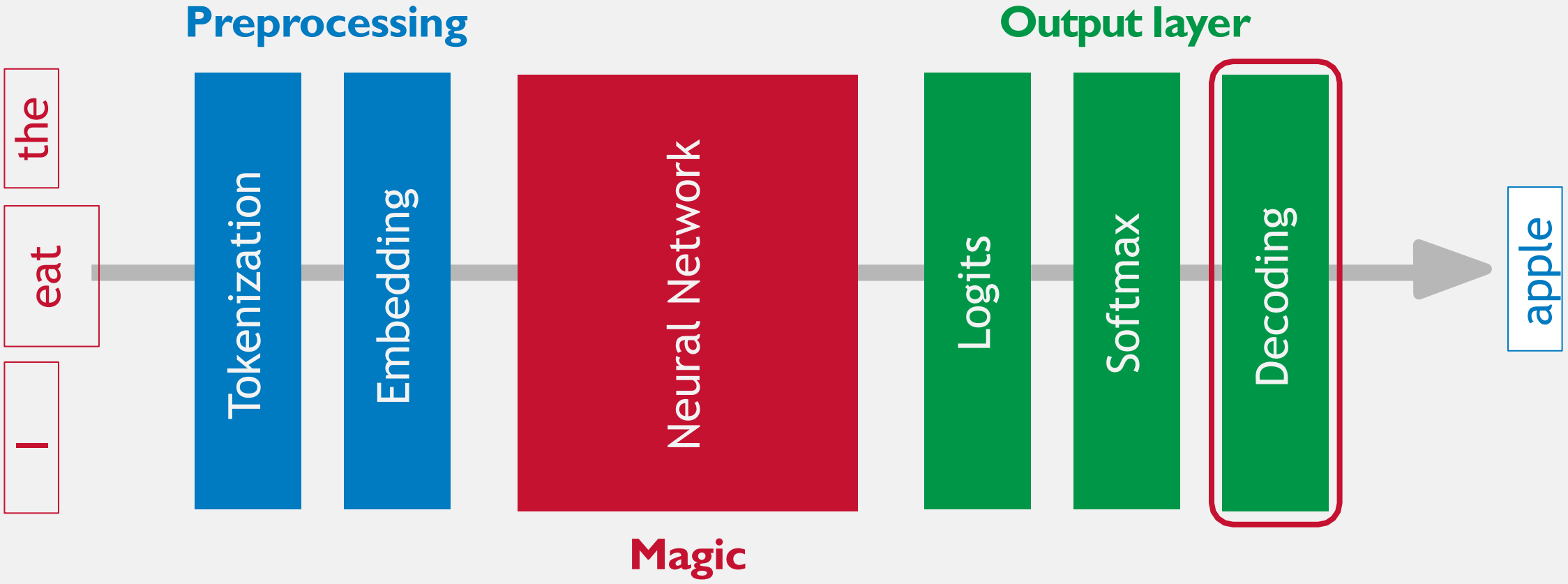
$$= -\sum_{t=1}^{T} \log \frac{\exp(\mathbf{E}\hat{y}_t[i^*])}{\sum_{j=1}^{|V|} \exp(\mathbf{E}\hat{y}_t[j^*])}$$

$$= -\sum_{t=1}^{T} \Big(\underbrace{\mathbf{E}\hat{y}_t[i^*]}_{①} - \underbrace{\log \textstyle\sum_{j=1}^{|V|} \exp(\mathbf{E}\hat{y}_t[j^*])}_{②} \Big)$$

① Increase the score of the correct word (bring the vector closer)

② Decrease the score of other words (push the vector further away)

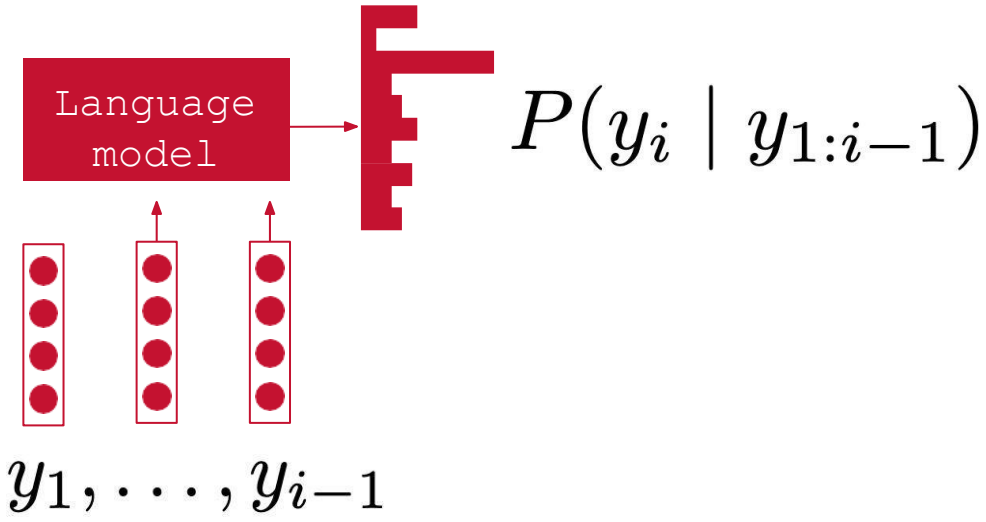
Maximize likelihood = Maximize similarity


Preprocessing
Output layer
the
eat
I
Tokenization
Embedding
Neural Network
Logits
Softmax
Decoding
apple
Magic

Decoding

How can we sample from $P(y_i | y_{1:i-1})$?

For generation, we need a sampling algorithm that selects a word given the predicted probability distribution.



Decoding

Option 1: $\underset{i}{\operatorname{argmax}} P(y_t = i \mid y_{1:t-1})$

Decoding

Option 1: $\underset{i}{\operatorname{argmax}} P(y_t = i \mid y_{1:t-1})$

Option 2: Randomly sample from the distribution returned by the model.

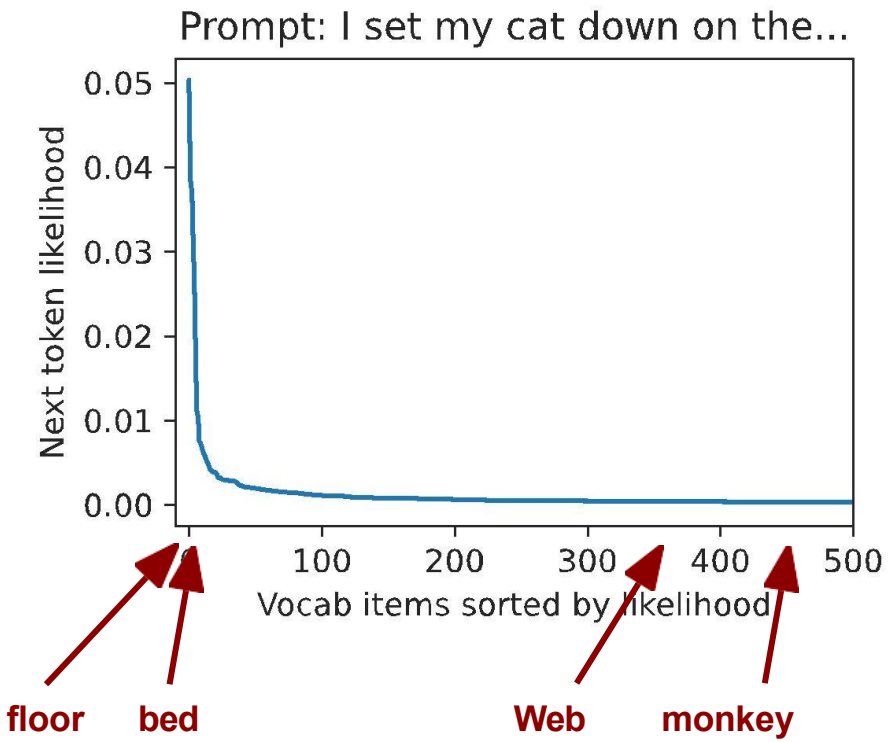
Decoding

Option 1: $\underset{i}{\operatorname{argmax}} P(y_t = i \mid y_{1:t-1})$

Option 2: Randomly sample from the distribution returned by the model.

Problem with Random Sampling

Most tokens in the vocabulary get assigned very low probabilities but cumulatively, choosing any one of these low-probability tokens is pretty likely. In the example on the right, there is over a 29% chance of choosing token `v` with $P(y_t=v) \leq 0.001$.



Solution: Modify the distribution returned by the model to make the tokens in the tail less likely.

Decoding

Option 1: $\underset{i}{\operatorname{argmax}} P(y_t = i \mid y_{1:t-1})$

Option 2: Randomly sample from the distribution returned by the model.

Option 3: Randomly sample with temperature T.

$$P(y_t = i \mid y_{1:t-1}) = \frac{\exp(z_i/T)}{\sum_{j=1}^{|V|} \exp(z_j/T)}$$

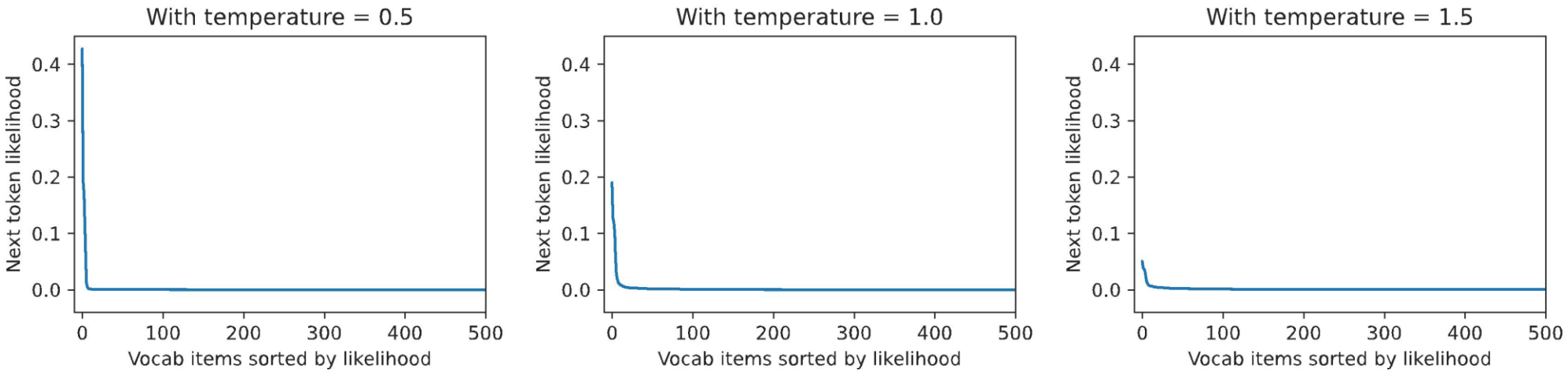
Decoding

Option 1: $\underset{i}{\operatorname{argmax}} P(y_t = i \mid y_{1:t-1})$

Option 2: Randomly sample from the distribution returned by the model.

Option 3: Randomly sample with temperature.

As $\mathtt{T}$ approaches 0, random sampling with temperature looks more and more like argmax.



Decoding

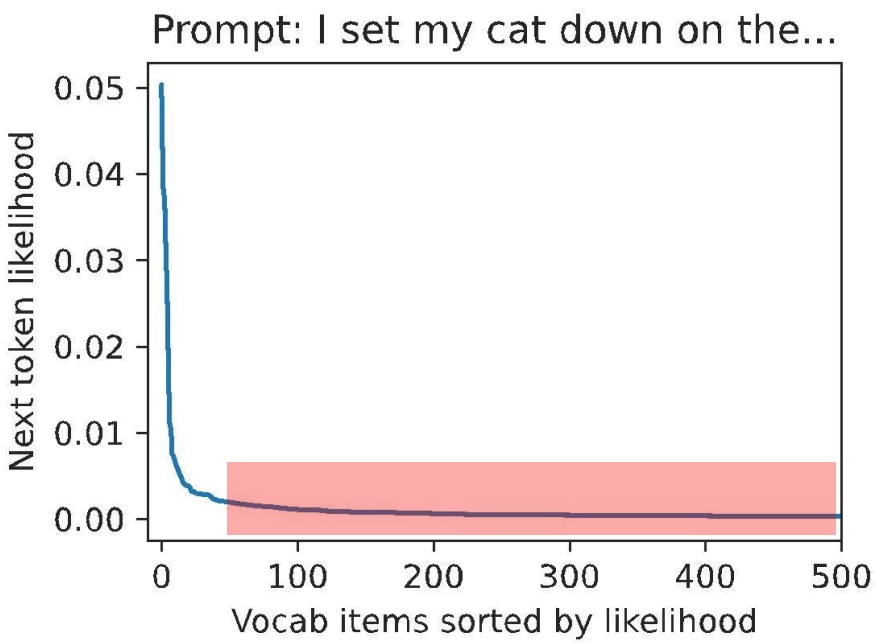
Option 1: $\underset{i}{\operatorname{argmax}} P(y_t = i \mid y_{1:t-1})$

Option 2: Randomly sample from the distribution returned by the model.

Option 3: Randomly sample with temperature.

Option 4: Introduce sparsity by reassigning all probability mass to the `k` most likely tokens. This is referred to as top-`k` sampling.

Usually `k` between 10 and 50 is selected.



Decoding

$$\underset{i}{\operatorname{argmax}} P(y_t = i \mid y_{1:t-1})$$

Option 1:

Option 2: Randomly sample from the distribution returned by the model.

Option 3: Randomly sample with temperature.

Option 4: Introduce sparsity by reassigning all probability mass to the k most likely tokens. This is referred to as top-k sampling.

Option 5: Introduce sparsity by reassigning all probability mass to the k_t tokens which form p of the probability mass.
At each step, k_t is chosen such that total probability of the k_t most likely tokens is no greater than the desired probability p. This is referred to as nucleus sampling.

$$\sum_{y \in V} P(y_i | y_{1:i-1}) \geq P$$

Decoding

$$\underset{i}{\operatorname{argmax}} P(y_t = i \mid y_{1:t-1})$$

Option 1:

Option 2: Randomly sample from the distribution returned by the model.

Option 3: Randomly sample with temperature.

Option 4: Introduce sparsity by reassigning all probability mass to the k most likely tokens. This is referred to as top-k sampling.

Option 5: Introduce sparsity by reassigning all probability mass to the k_t tokens which form p of the probability mass.
At each step, k_t is chosen such that total probability of the k_t most likely tokens is no greater than the desired probability p. This is referred to as nucleus sampling.

Option 6: Use some version of beam search.

Beam Search

Assumption: The best possible sequence to generate is the one with highest overall sequence likelihood (according to the model).

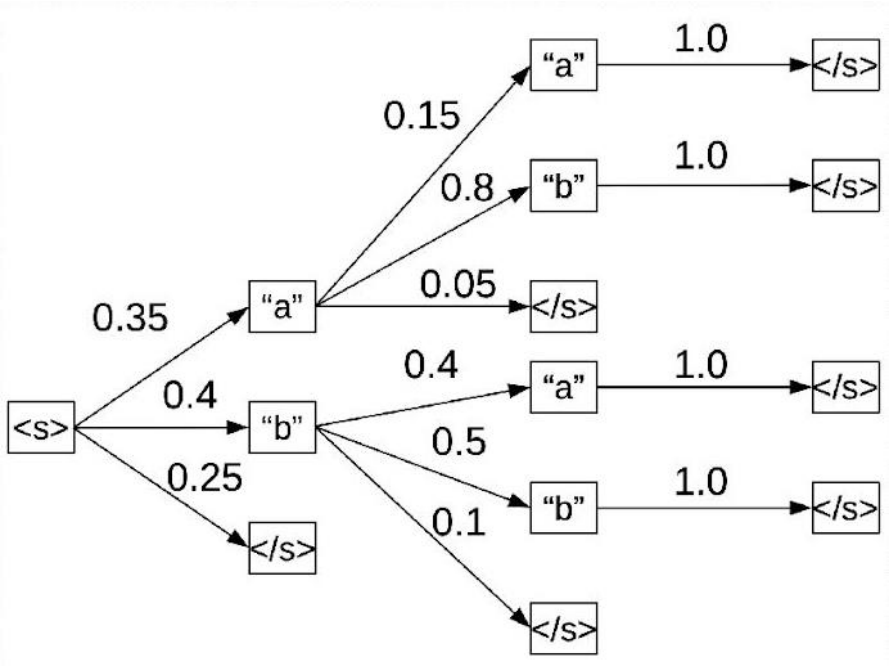
It is computationally intractable to search all possible sequences for the most likely one, so instead we use beam search.

Beam search is a search algorithm that approximates finding the overall most likely sequence to generate.

Beam Search: Example

$$V = \{a, b, < S >\}$$

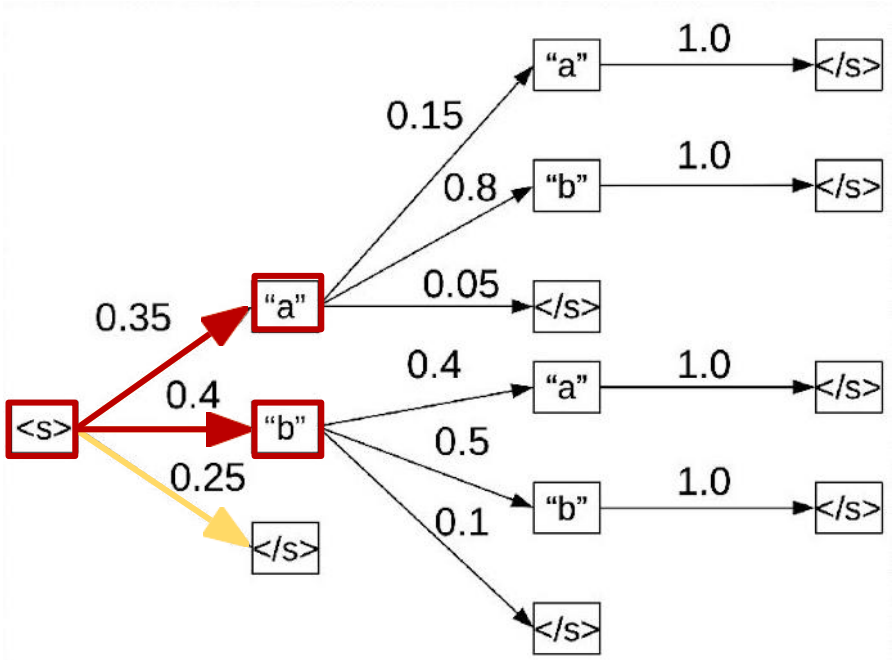
Numbers above each edge are the transition probabilities $P(y_t \mid y_{1:t-1})$



Beam Search: Example

$$V = \{a, b, < S >\}$$

Numbers above each edge are the transition probabilities $P(y_t \mid y_{1:t-1})$

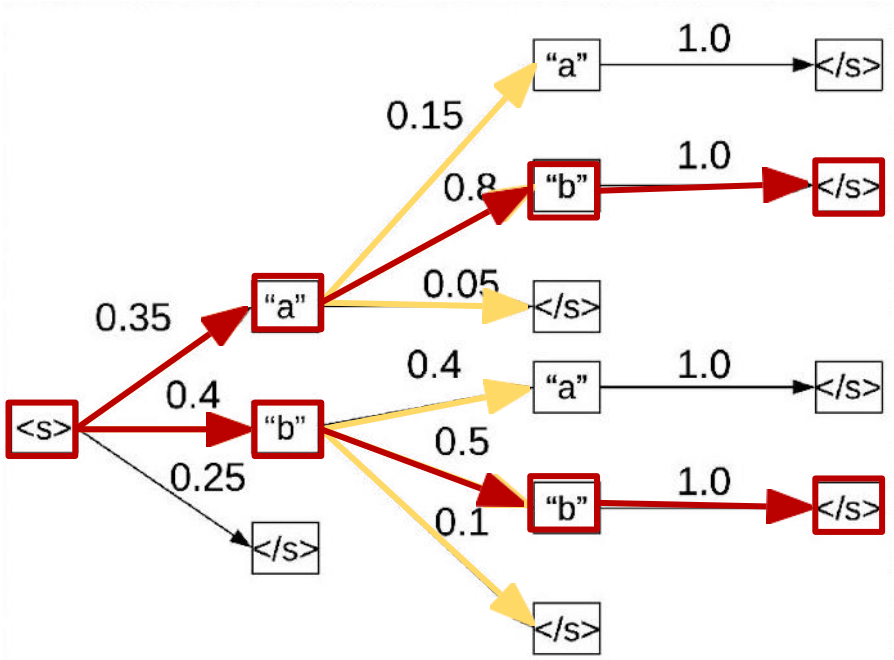


Let's perform beam search with a beam size of 2.

Beam Search: Example

$$V = \{a, b, < S >\}$$

Numbers above each edge are the transition probabilities $P(y_t \mid y_{1:t-1})$

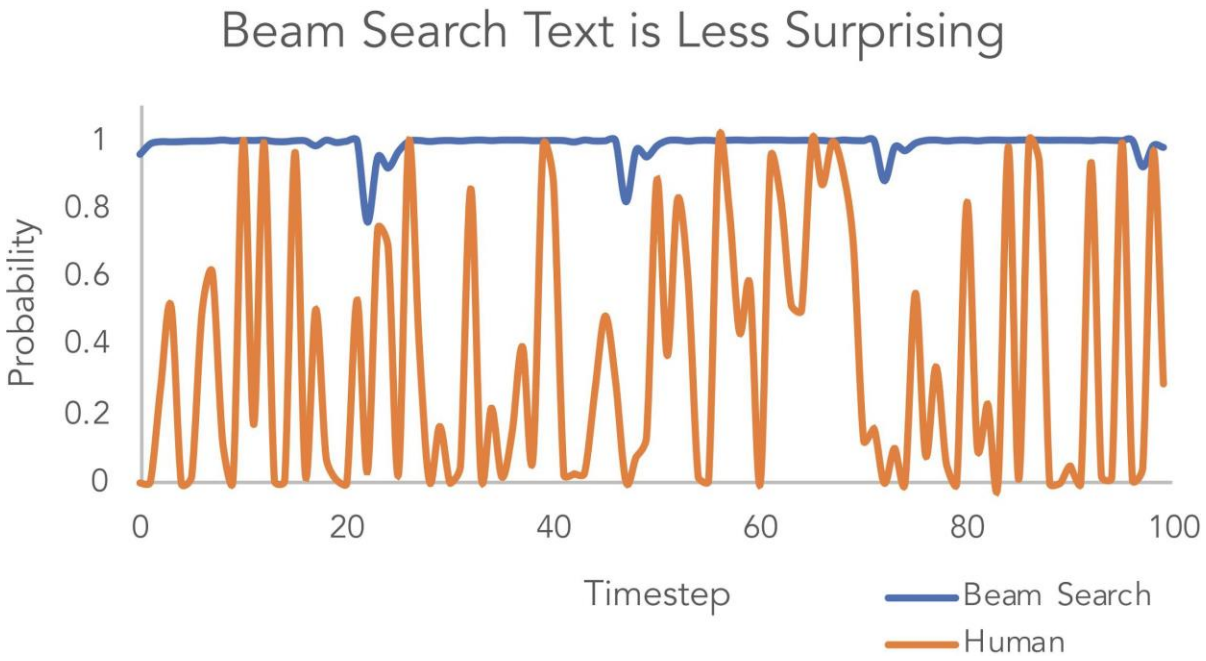


The paths that are still open at any step of the beam search algorithm are called beams.

Possible Problems with High Probability Approaches

It turns out for open-ended tasks like dialog or story generation, optimizing for the sequence with the highest possible $P(x_1,\ldots,x_T)$ may not always be the best idea.

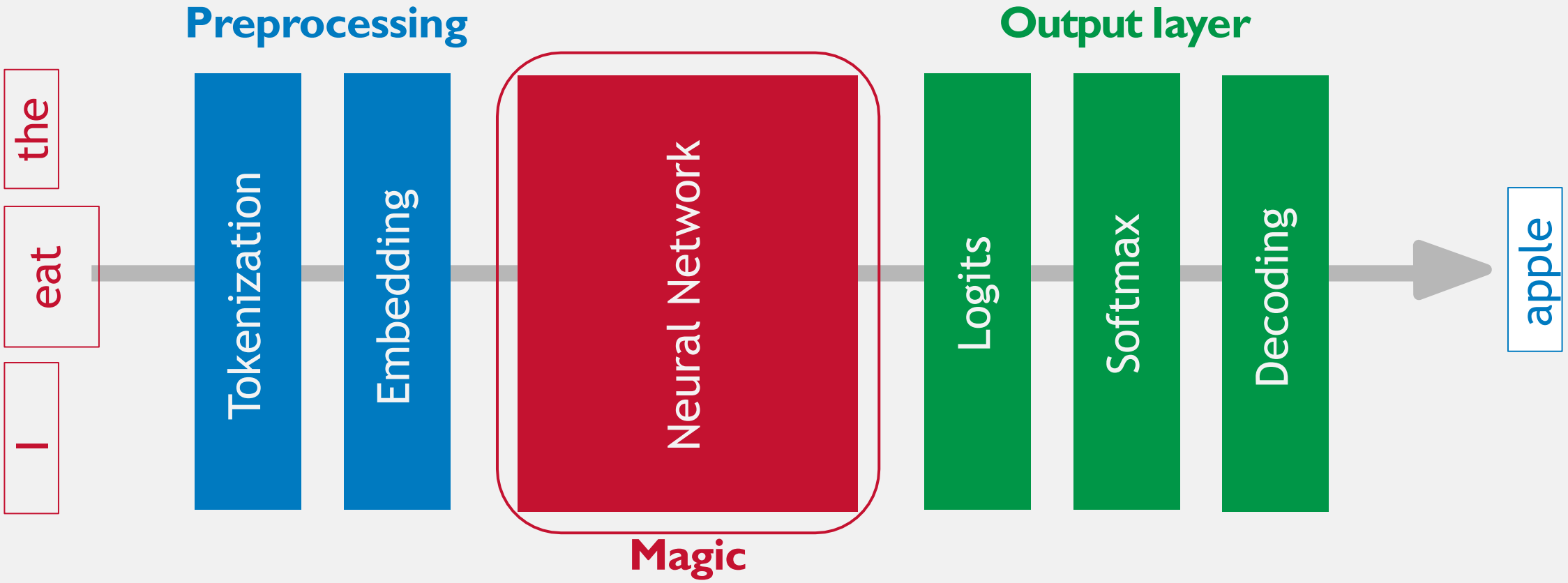
- Beam search generates text that is much more likely than human-written text.
- When sequence likelihood is too high humans rate text as bad.



Other Generation Parameters

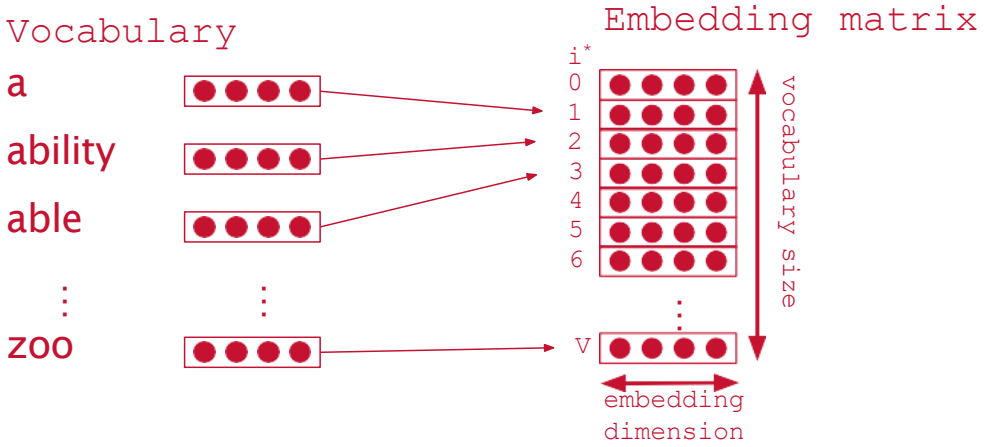
- Frequency penalty: Reduce the likelihood of the model generating a token based on how often it has occurred already.
  - The more likely a token has occurred, the less likely it will occur in the future.
- Presence penalty: Reduce the likelihood the model generates a token based on whether or not it has occurred already.
  - If a token occurs any number of times it will be less likely to occur in the future.
- Stopping criteria
  - Stop after generating k tokens
  - Stop when a certain token is generated (for example, a period or a newline)

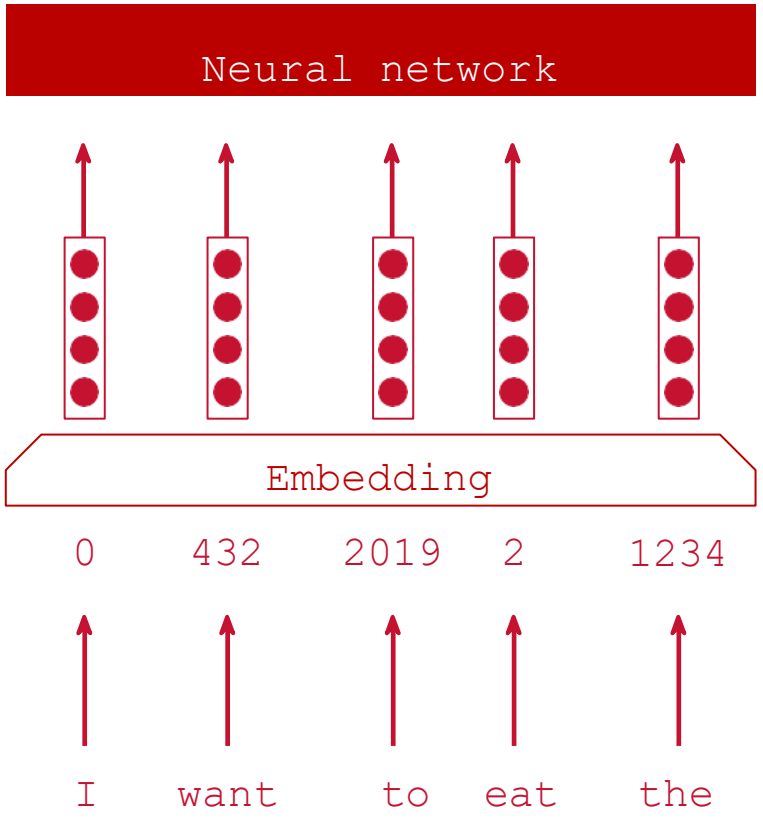
Neural Language Modeling


I
eat
the
Preprocessing
Tokenization
Embedding
Neural Network
Magic
Output layer
Logits
Softmax
Decoding
apple

NN Input → NN Output

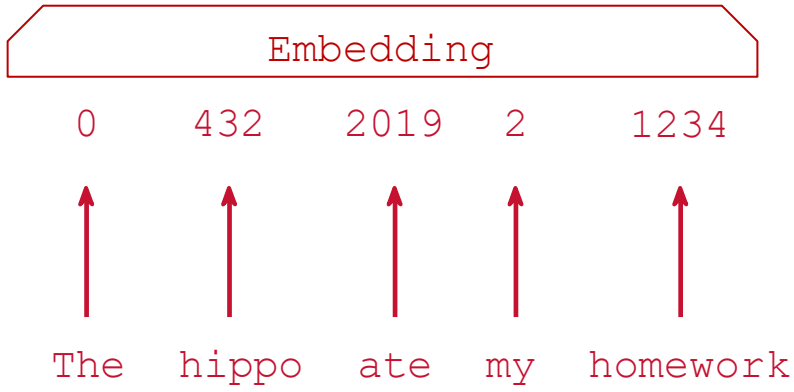
- The LM takes as input the vectors of each token in the input sequence.

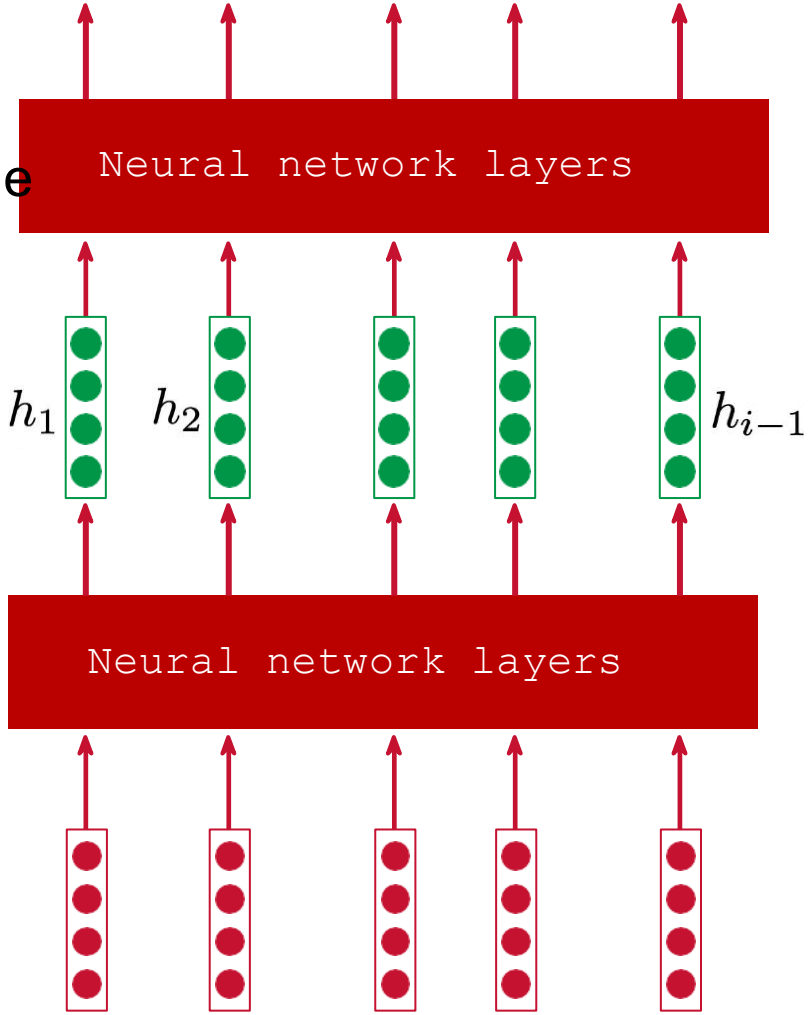




NN Input → NN Output

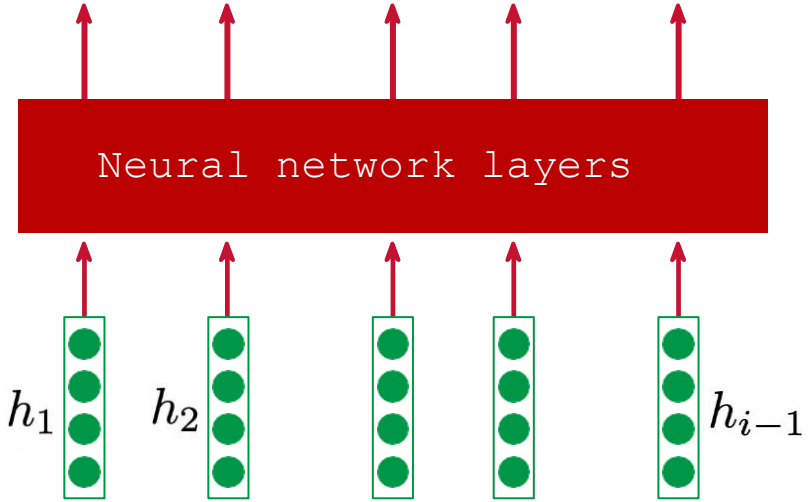
- The LM takes as input the vectors of each token in the input sequence.
- Vectors called hidden states are repeatedly produced and passed on.

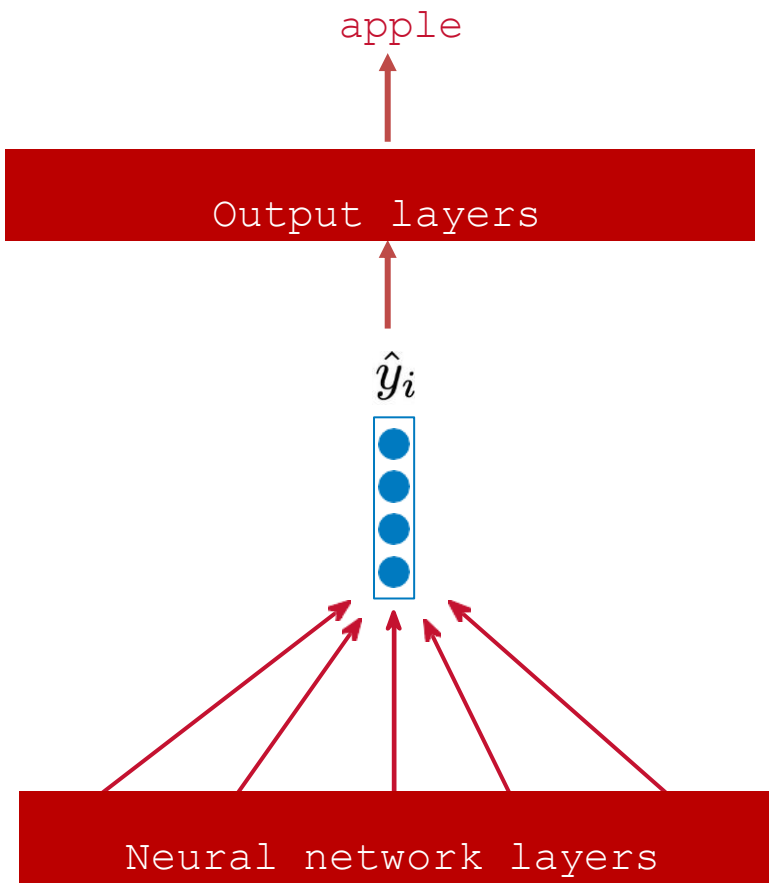




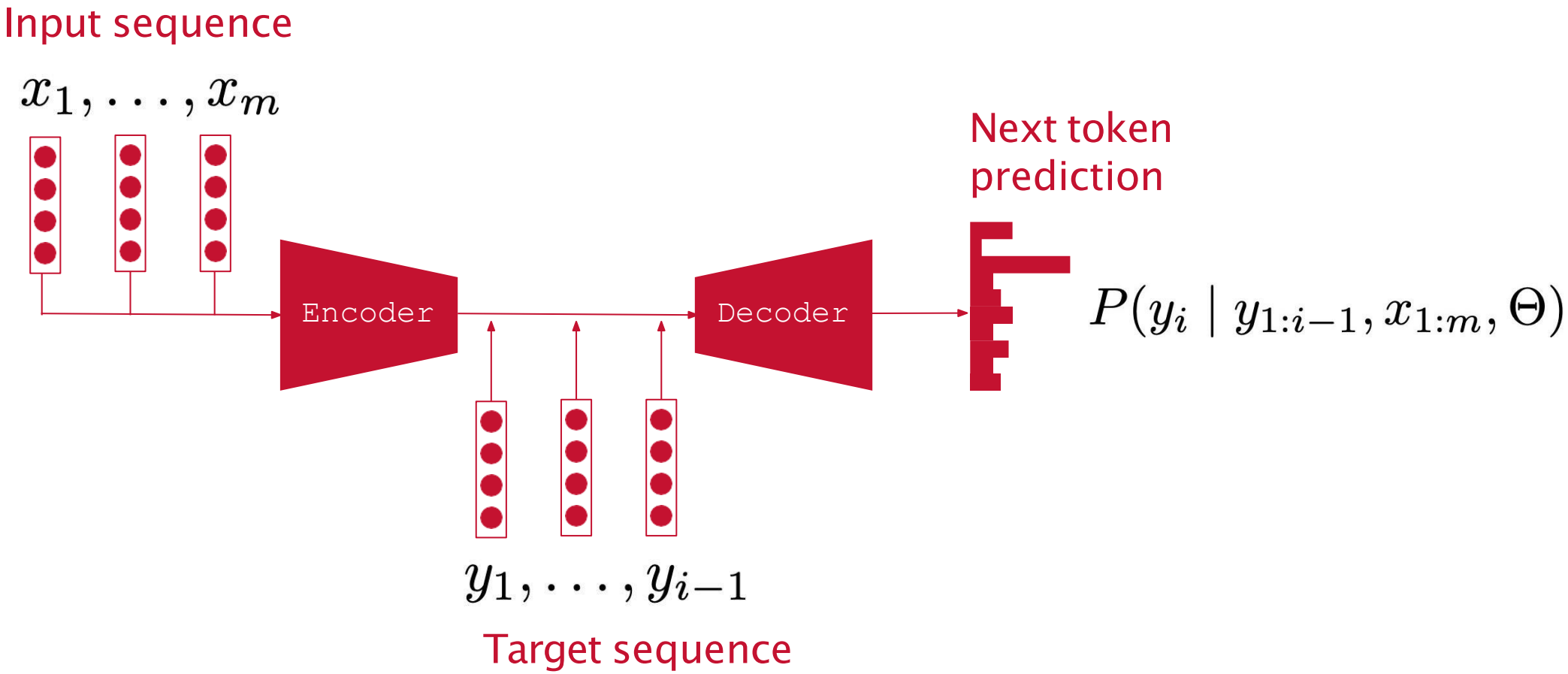
NN Input → NN Output

- The LM takes as input the vectors of each token in the input sequence.
- Embeddings called hidden states are repeatedly produced and passed on.
- An output embedding representing the generated token is produced.

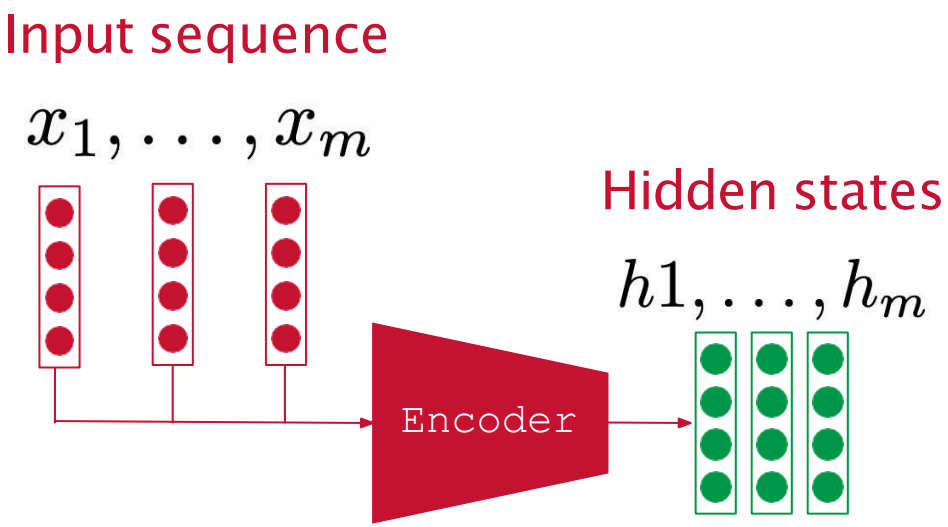




Encoder-Decoder Architecture



Encoder

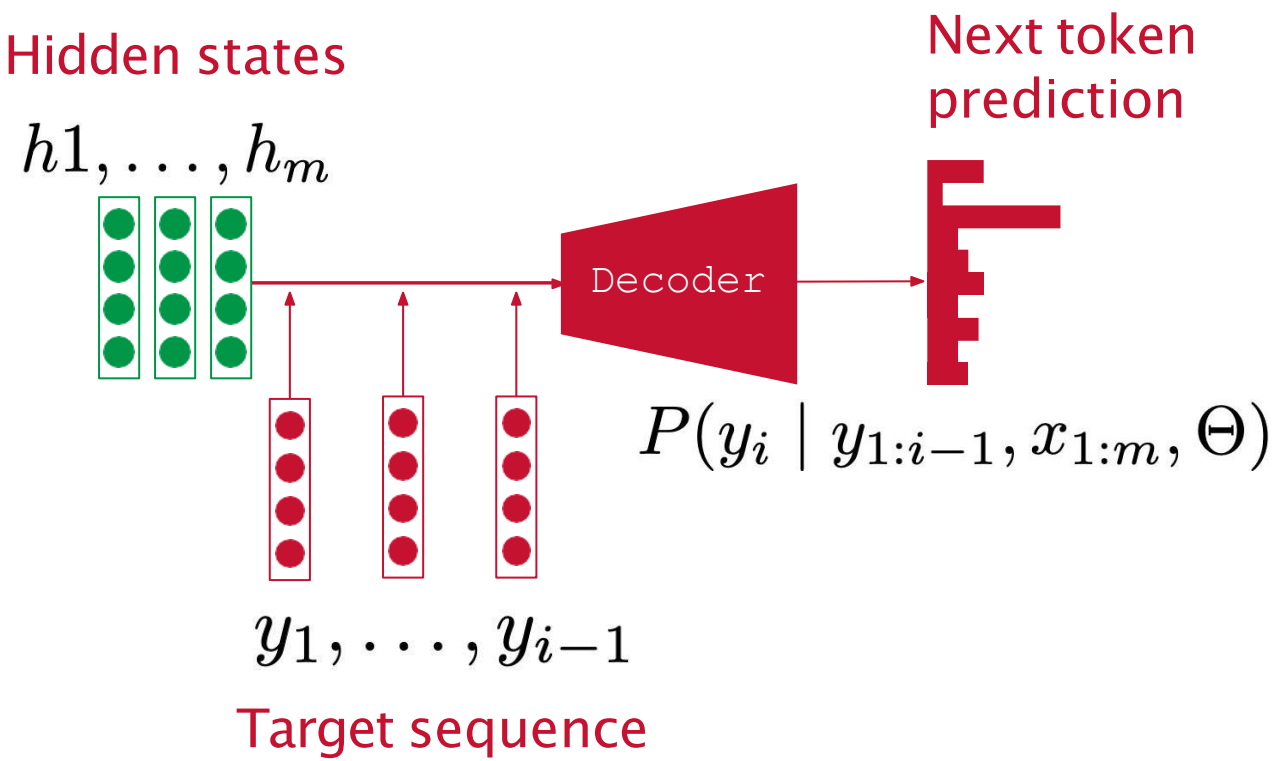


The input sequence is of variable length.

The output is is a vector summary of the input sequence.

The goal is to encode the input sequence so that it can be utilized by a decoder.

Decoder

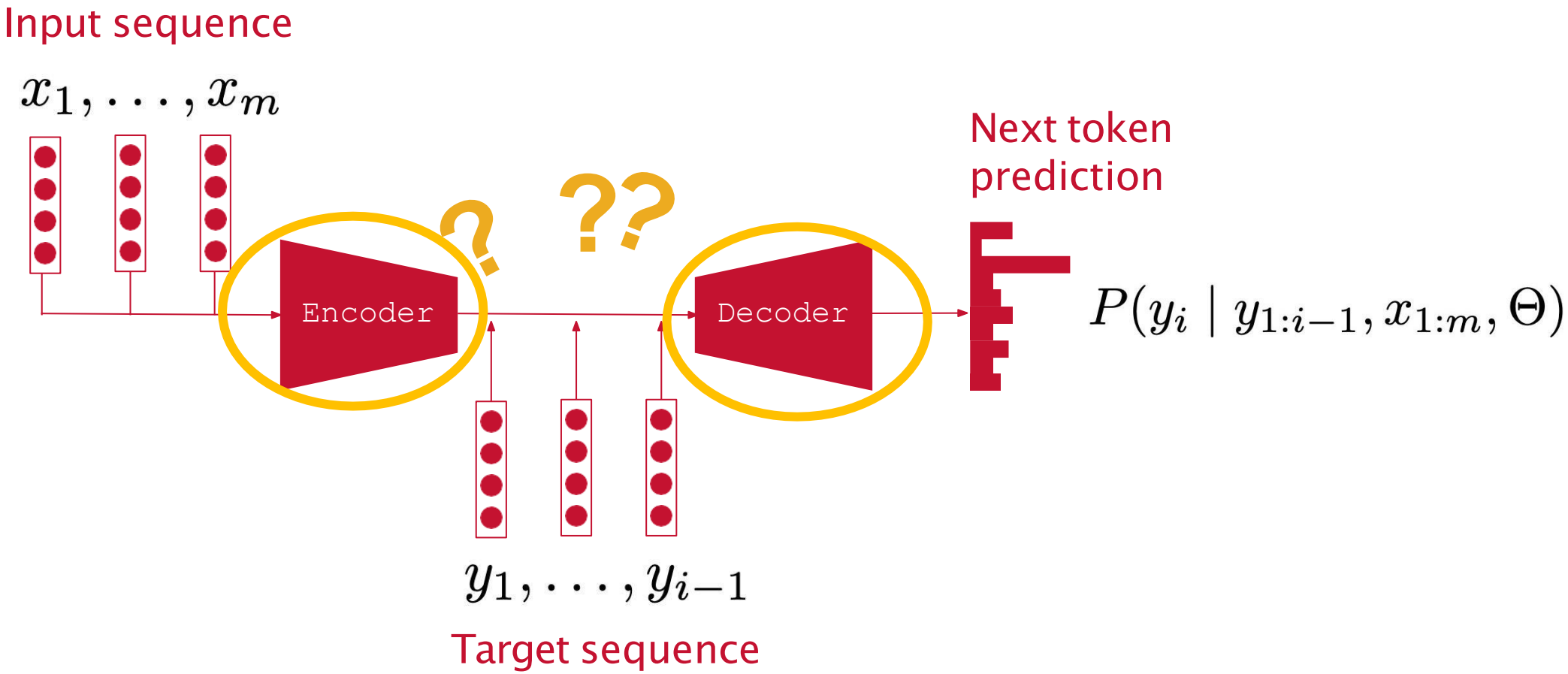


The target sequence is of variable length.

Decoder processes the target sequence token at a time and predicts the next token.

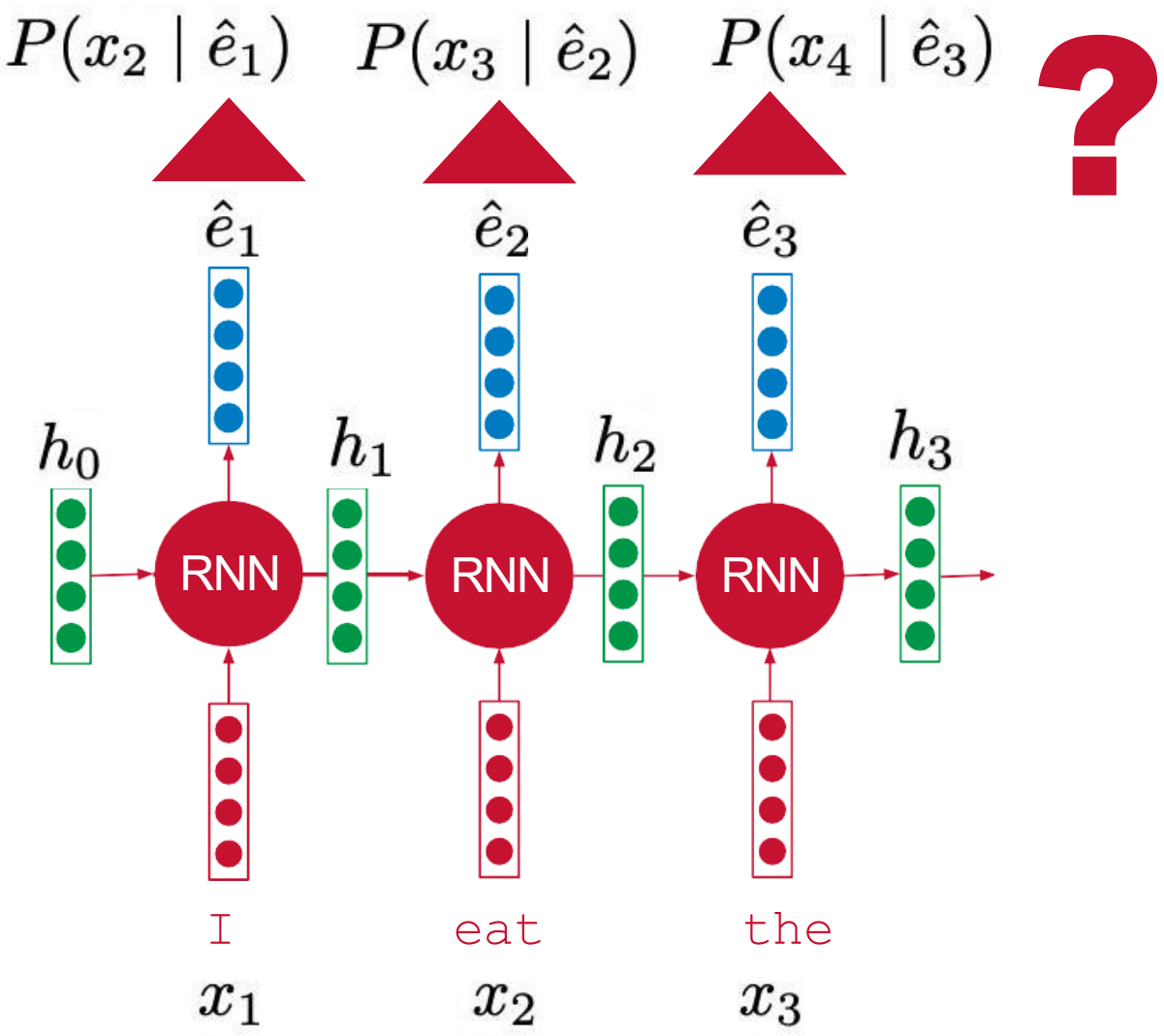
It utilizes the hidden state(s) produced by the encoder as context.

What are the encoder and decoder?



RNNs as Language Models

Recurrent Neural Network



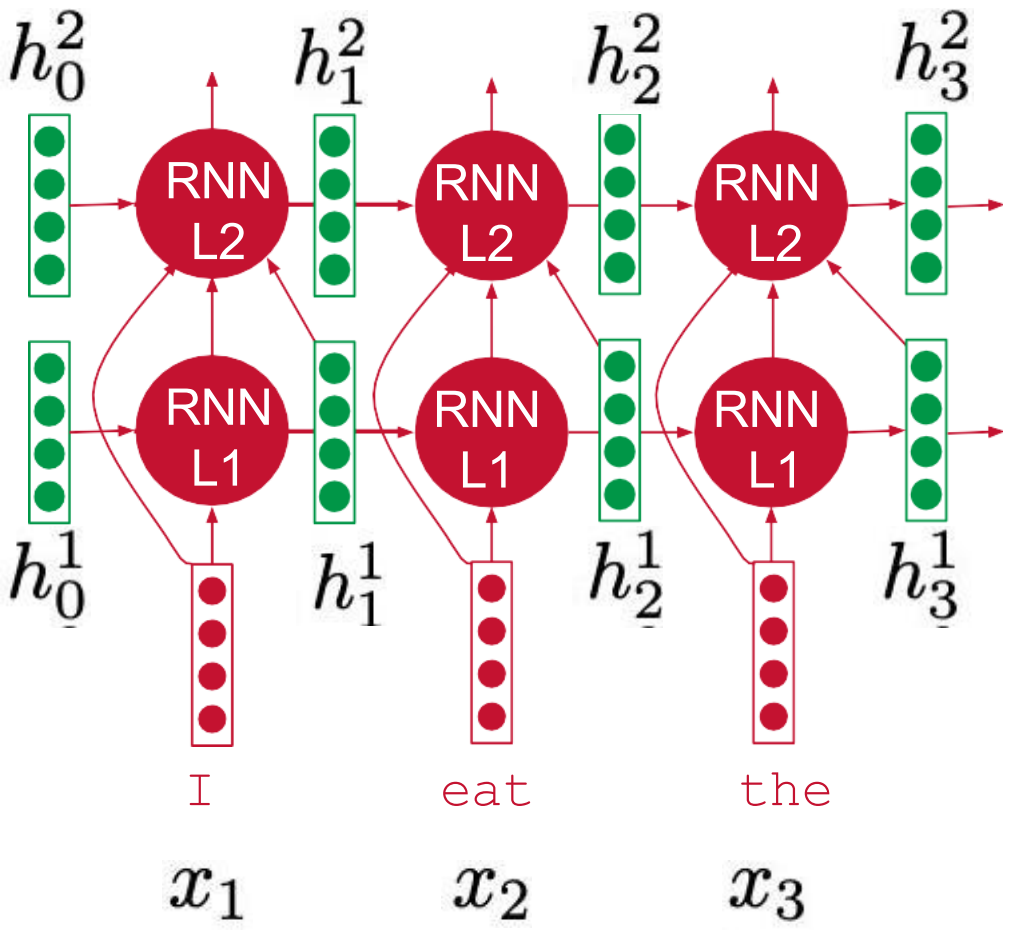
1. We input a sequence of embeddings.

2. A recurrent unit (aka RNN) receives the previous hidden state and the embedding for the token being processed.

3. The RNN outputs a predicted embedding (the next token) and an updated hidden state.

4. The first hidden state is typically initialized with a zero vector.

Recurrent Neural Network



Computing the next hidden state:

For the first layer:

$$h_t^1 = RNN\left(W_{ih^1}x_t + W_{h^1h^1}h_{t-1}^1 + b_h^1\right)$$

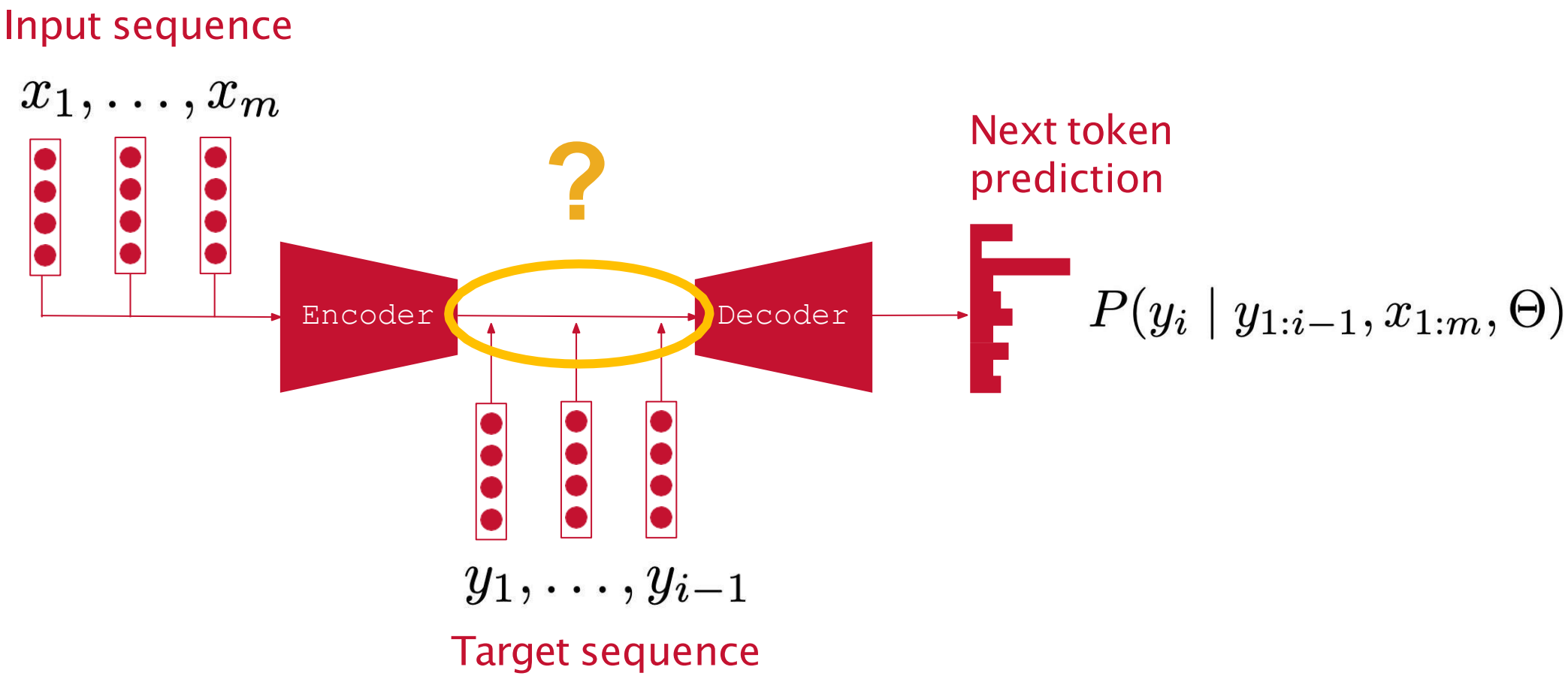
For all subsequent layers:

$$h_t^l = RNN\left(W_{ih^l}x_t + W_{h^{l-1}h^l}h_t^{l-1} + W_{h^lh^l}h_{t-1}^l + b_h^l\right)$$

Each of the b and W are learned bias and weight matrices.

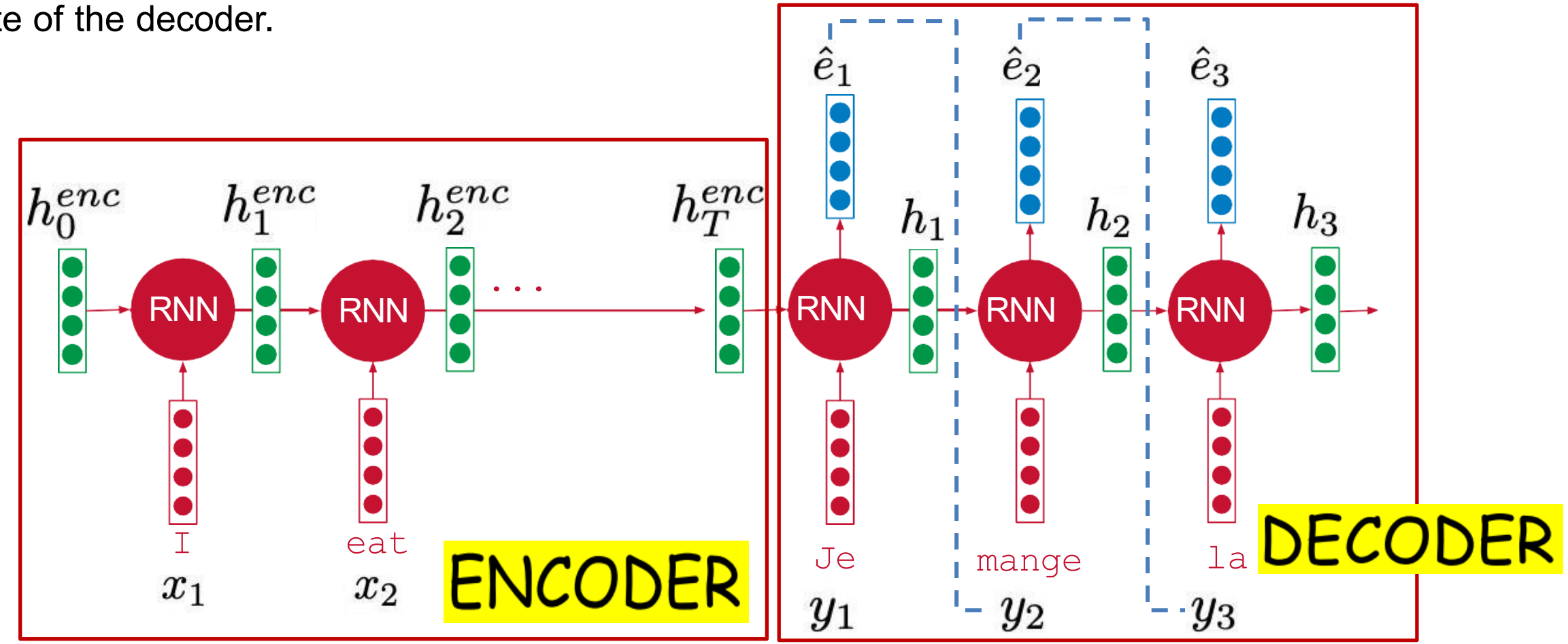
Recurrent Neural Network

How do RNN language models connect the encoder with the decoder?



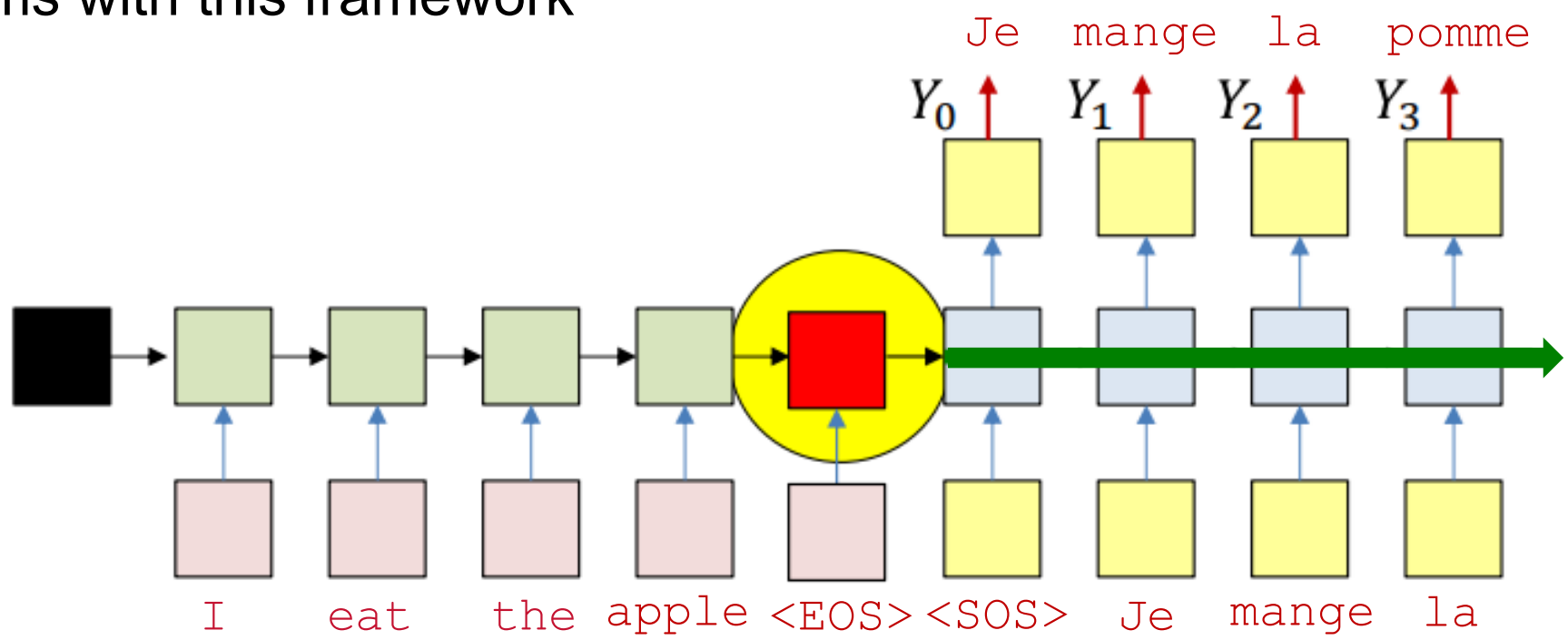
Recurrent Neural Network

Simplest approach: Use the final hidden state from the encoder to initialize the first hidden state of the decoder.



Recurrent Neural Network

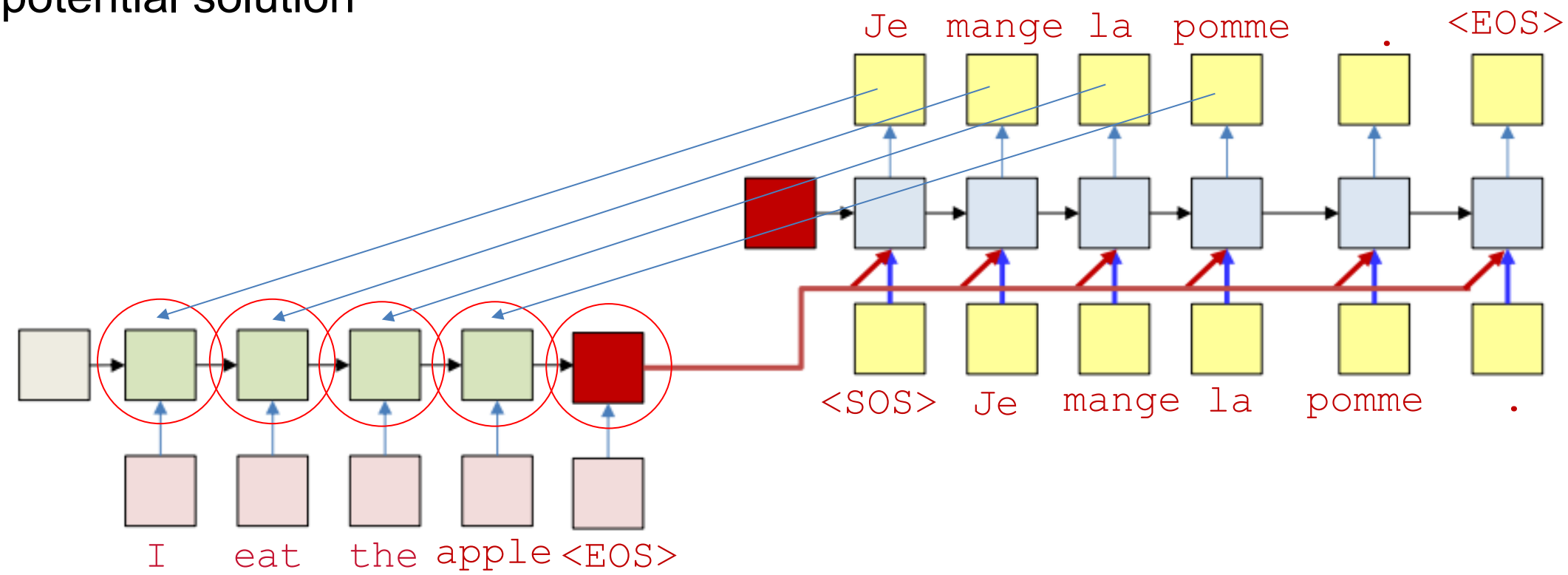
Problems with this framework



- All the information in the input is encoded in the final decoder hidden state.
  - Subsequently, this information is only carried through by recurrence
- The information in the final encoder hidden state will get “diluted” by the recurrence in the decoder.
  - At later decoder time steps the hidden state has a large contributions from the decoder itself, masking the contribution of the encoder hidden state

Recurrent Neural Network

A potential solution

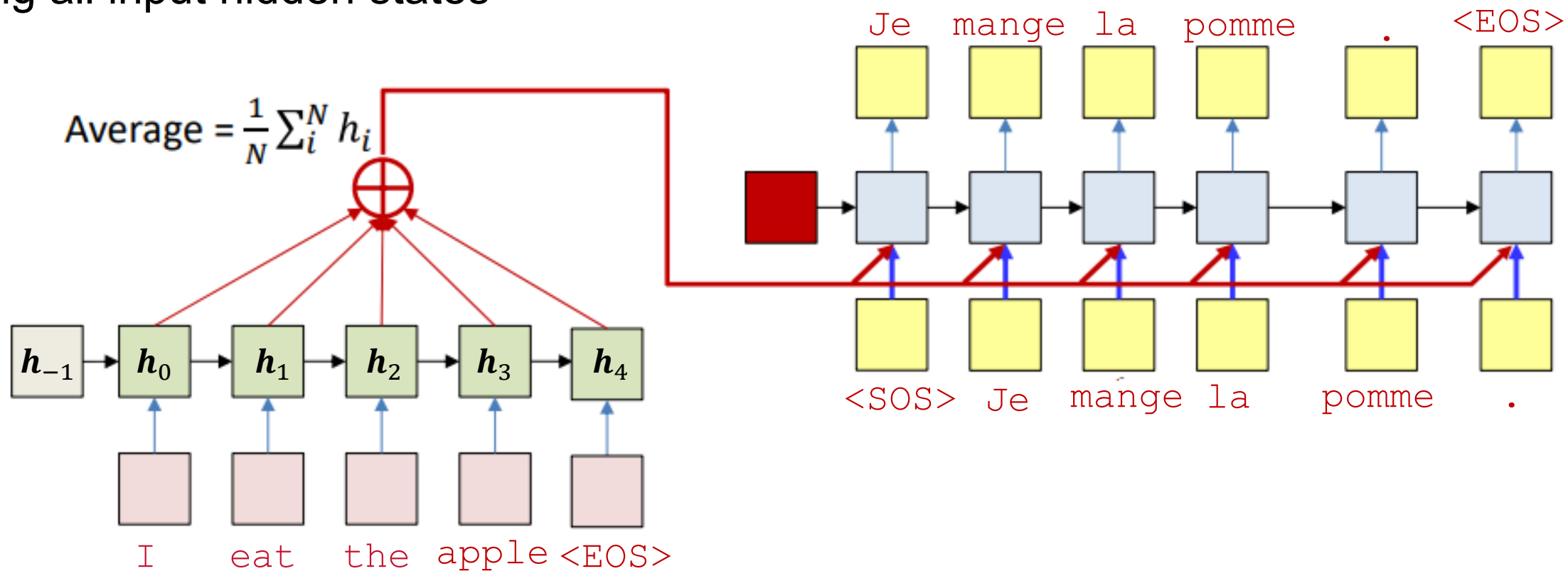


- Have the encoder representation feed directly to all decoder states as input
- The initial decoder hidden state is now separate from the encoder output
  - And may be a learnable parameter

Attention Mechanisms

Attention mechanism

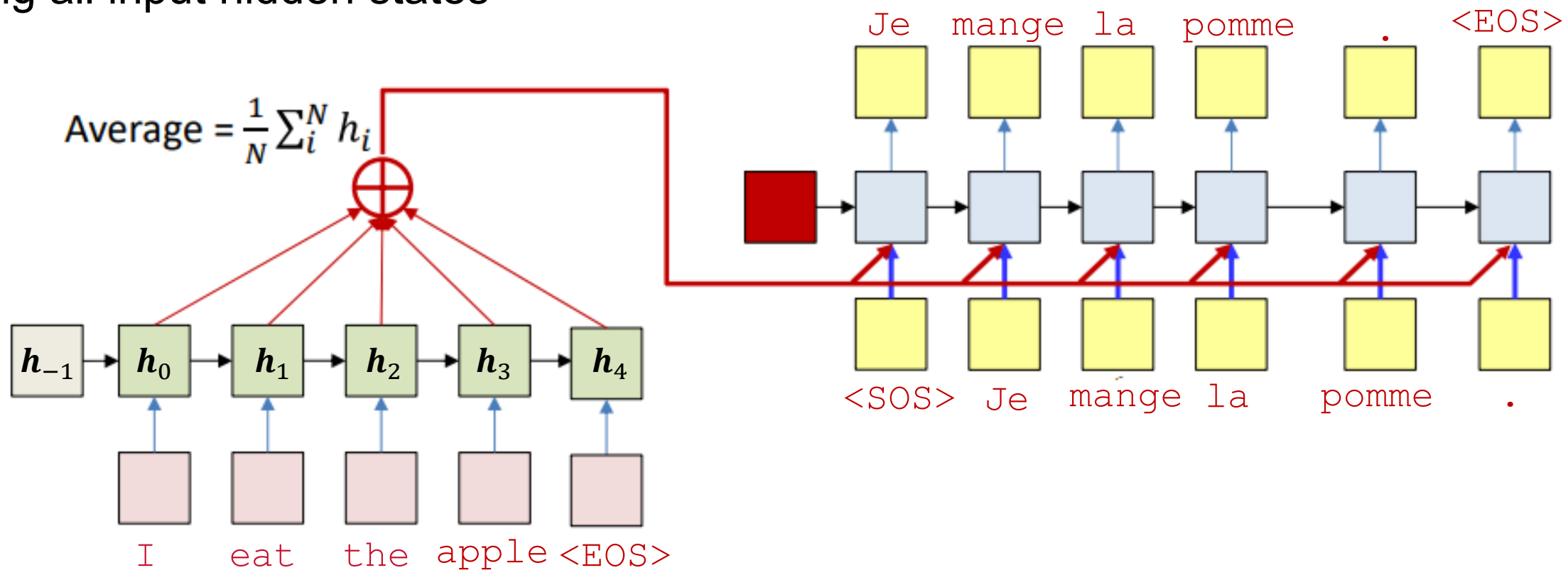
Using all input hidden states



- Attention mechanism: Compute the average of all encoder hidden states
- Input this average to every stage of the decoder

Attention mechanism

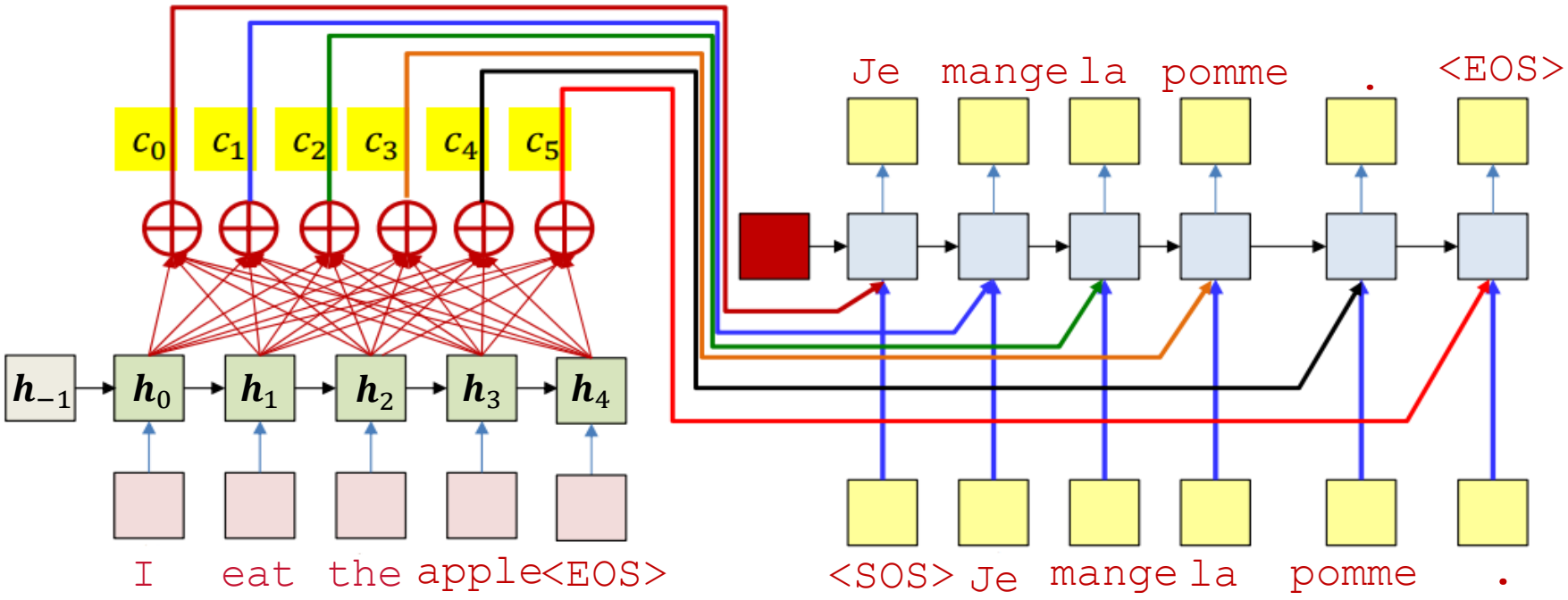
Using all input hidden states



- Problem: The average applies the same weight to every input
- It supplies the same average to every output word
- In practice, different outputs may be related to different inputs
  - E.g. “Je” is most related to “I”, and “mange” and “pomme” are both most related to “eat”

Attention mechanism

Using all input hidden states



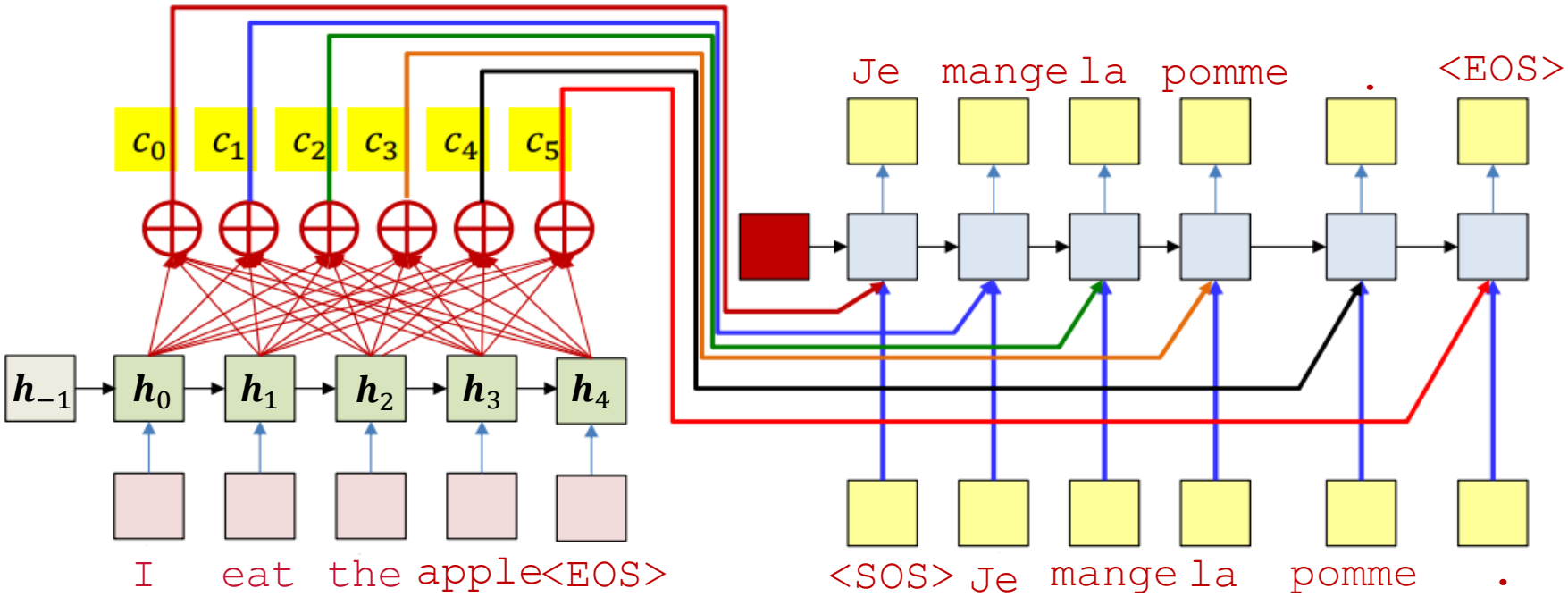
Solution: Use a different weighted average for each output word

– The weighted average provided for the kth output word is:

$$c_t = \frac{1}{N}\sum_{i}^{N} w_i(t) h_i$$

"Context" vector which captures the relevant input context for each output

Attention mechanism

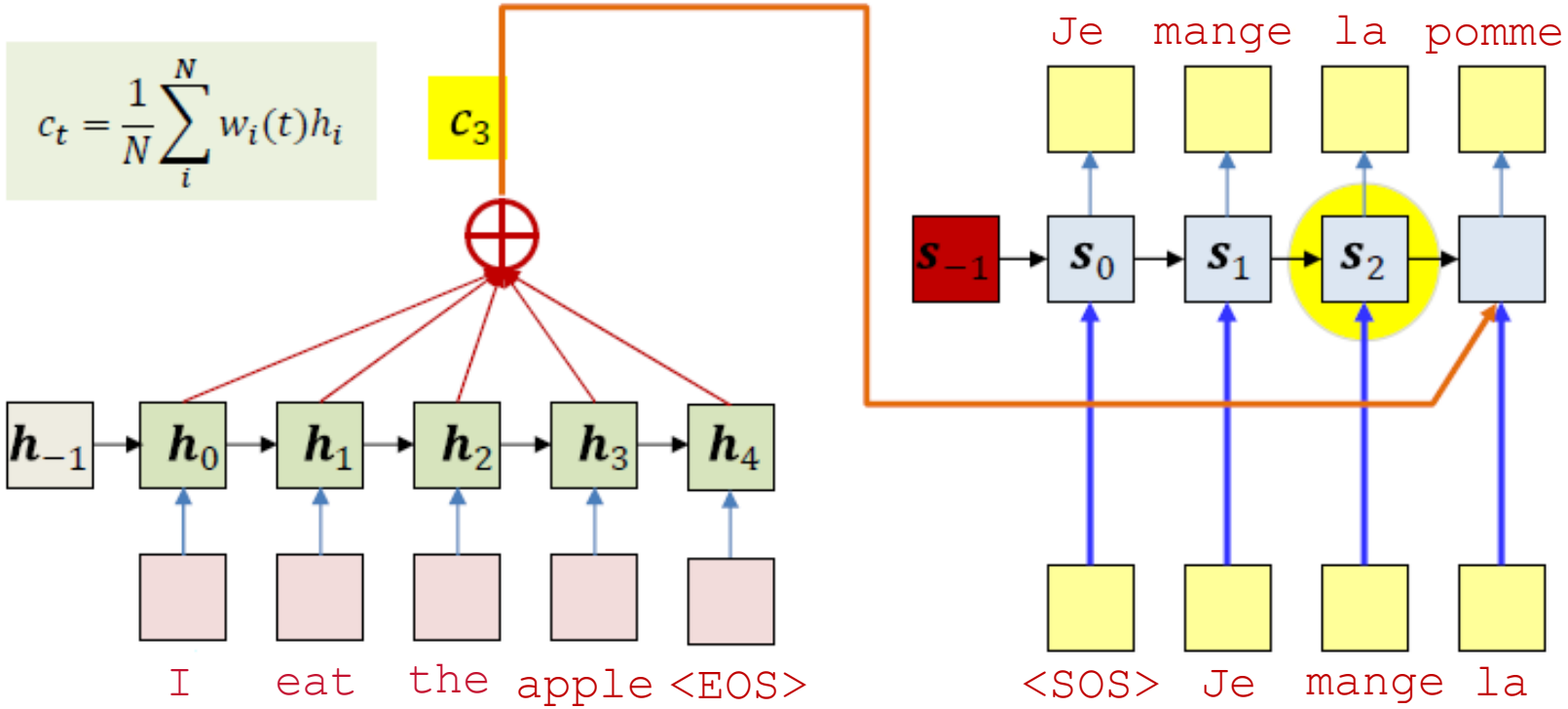


- We hope that the weights can somehow be made to "focus" on the right input word
  – E.g., when predicting the word "mange", $w_1(1)$, the weight for "apple" must be high while the rest must be low

How do we generate such weights??

- **Attention weights:** The weights $w_i(t)$ are dynamically computed as functions of decoder state

Attention mechanism



- The "attention" weights $w_i(t)$, at time t must be computed from available information at time t
- The primary information is s_{t-1} (the state at time $t-1$)

$$w_i(t) = a(\boldsymbol{h}_i, \boldsymbol{s}_{t-1})$$

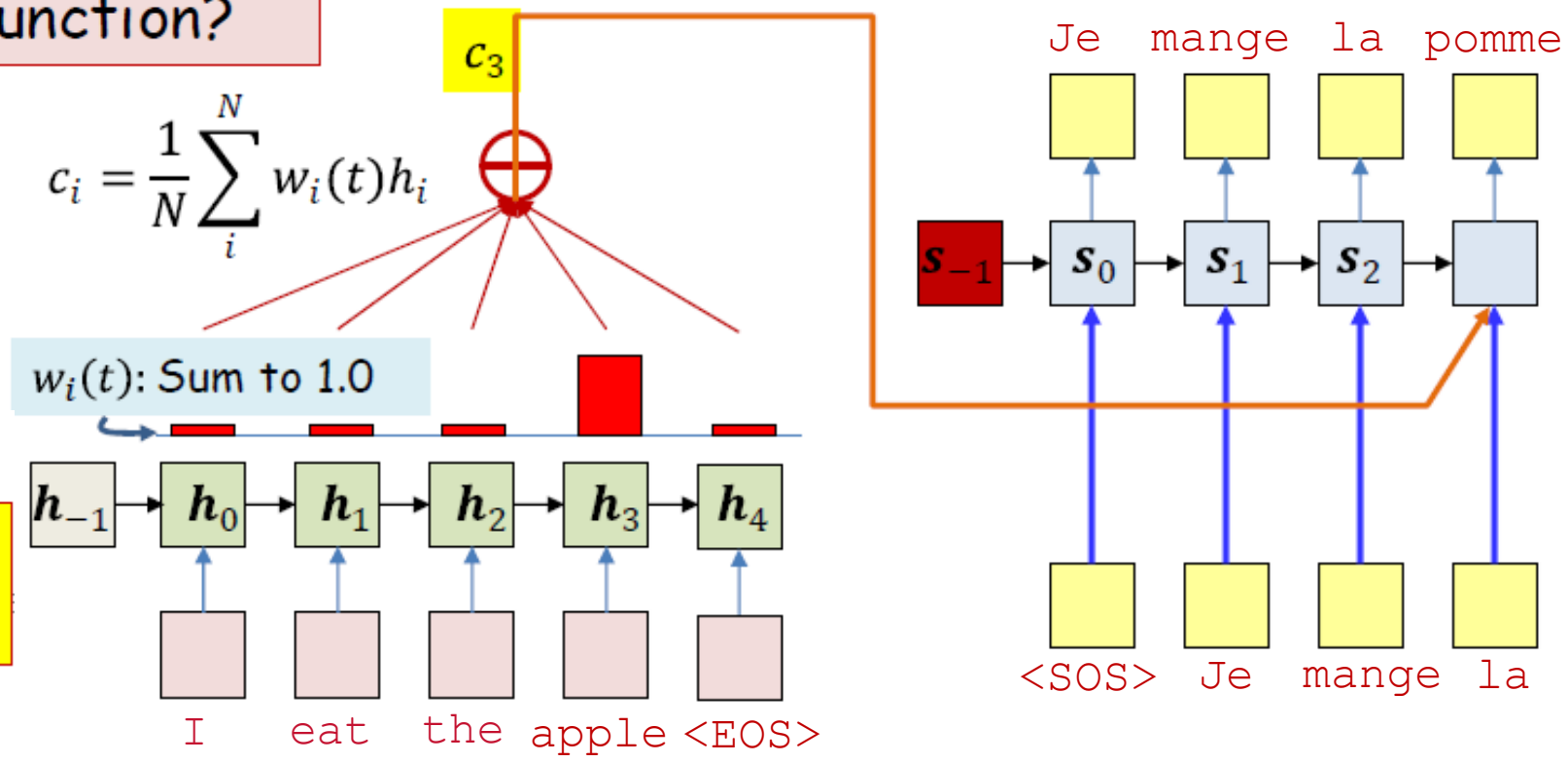
Attention mechanism

What is this function?

$$e_i(t) = g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

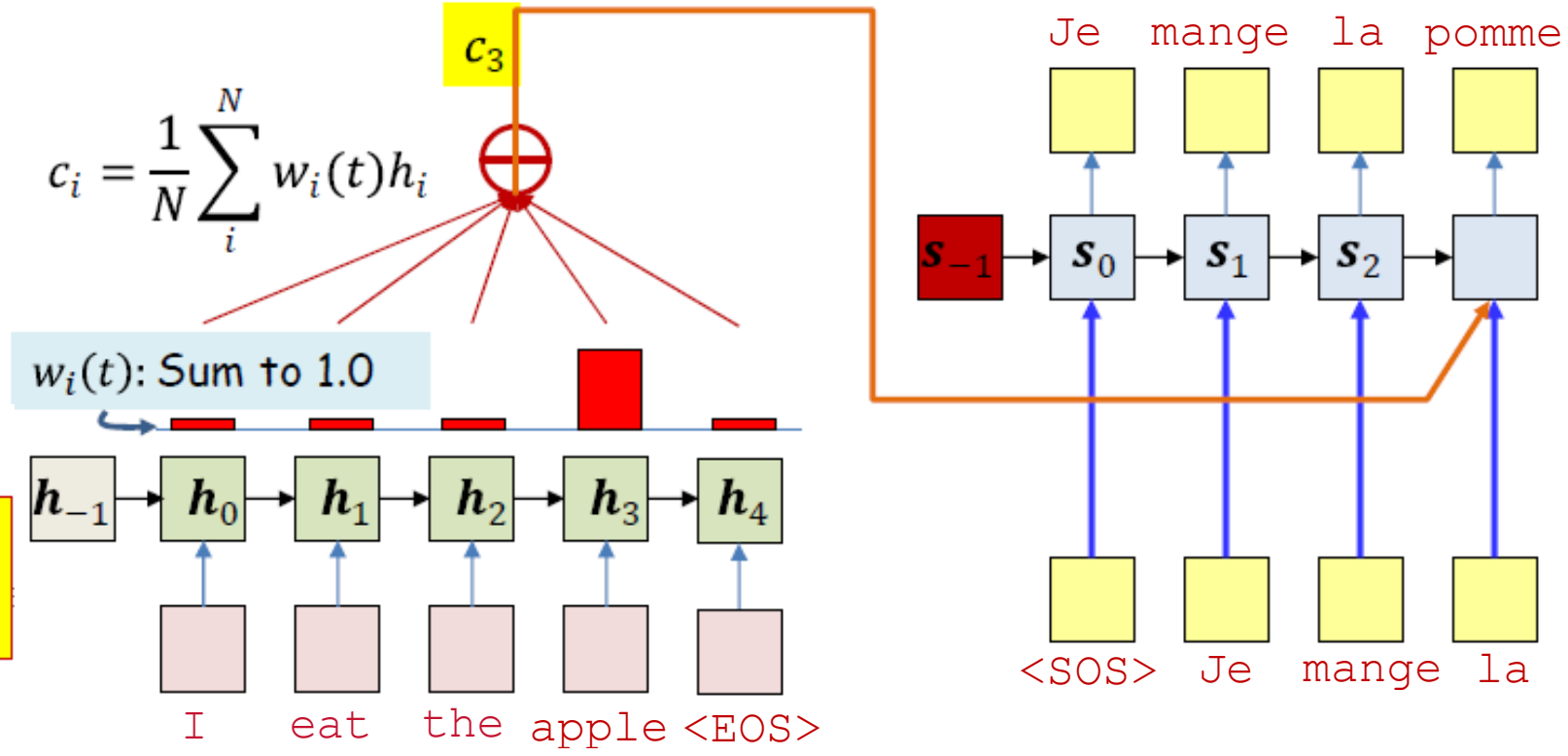
$$c_i = \frac{1}{N}\sum_i^N w_i(t) h_i$$



- The weights $w_i(t)$ must be positive and sum to 1.0
  - I.e. be a distribution
  - Ideally, they must be high for the most relevant inputs for the ith output and low elsewhere

Attention mechanism

$$c_i = \frac{1}{N}\sum_i^N w_i(t) h_i$$



$$e_i(t) = g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

- The typical options for $g()$

$$g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1}) = \boldsymbol{h}_i^T \boldsymbol{s}_{t-1}$$

$$g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1}) = \boldsymbol{h}_i^T \boldsymbol{W}_g \boldsymbol{s}_{t-1}$$

$$g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1}) = \boldsymbol{v}_g^T \tanh\left(\boldsymbol{W}_g \begin{bmatrix} \boldsymbol{h}_i \\ \boldsymbol{s}_{t-1} \end{bmatrix}\right)$$

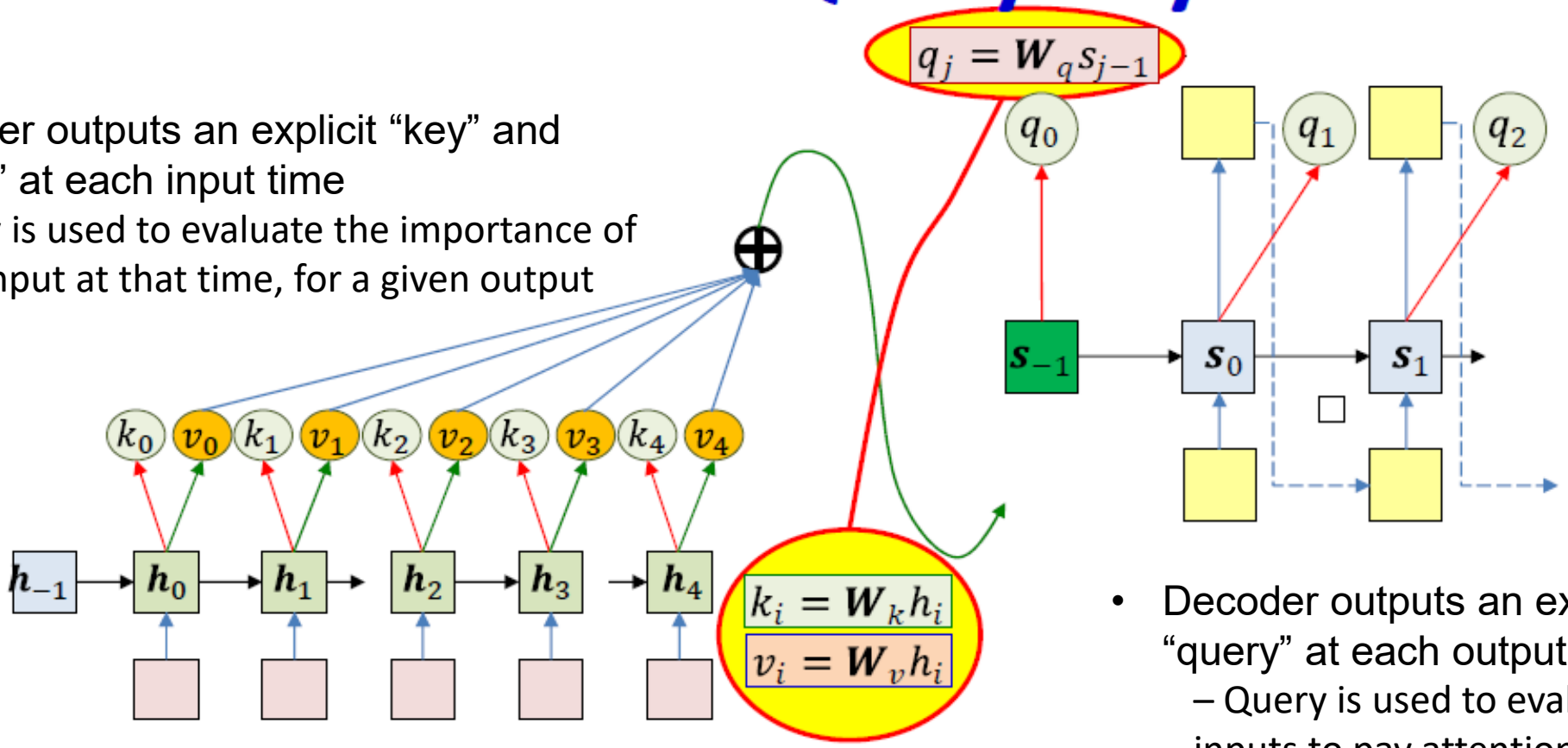
$$g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1}) = MLP([\boldsymbol{h}_i, \boldsymbol{s}_{t-1}])$$

variables in red can be learned

Attention mechanism

Modification: Query key value

- Encoder outputs an explicit "key" and "value" at each input time
  - Key is used to evaluate the importance of the input at that time, for a given output



- Decoder outputs an explicit "query" at each output time
  - Query is used to evaluate which inputs to pay attention to

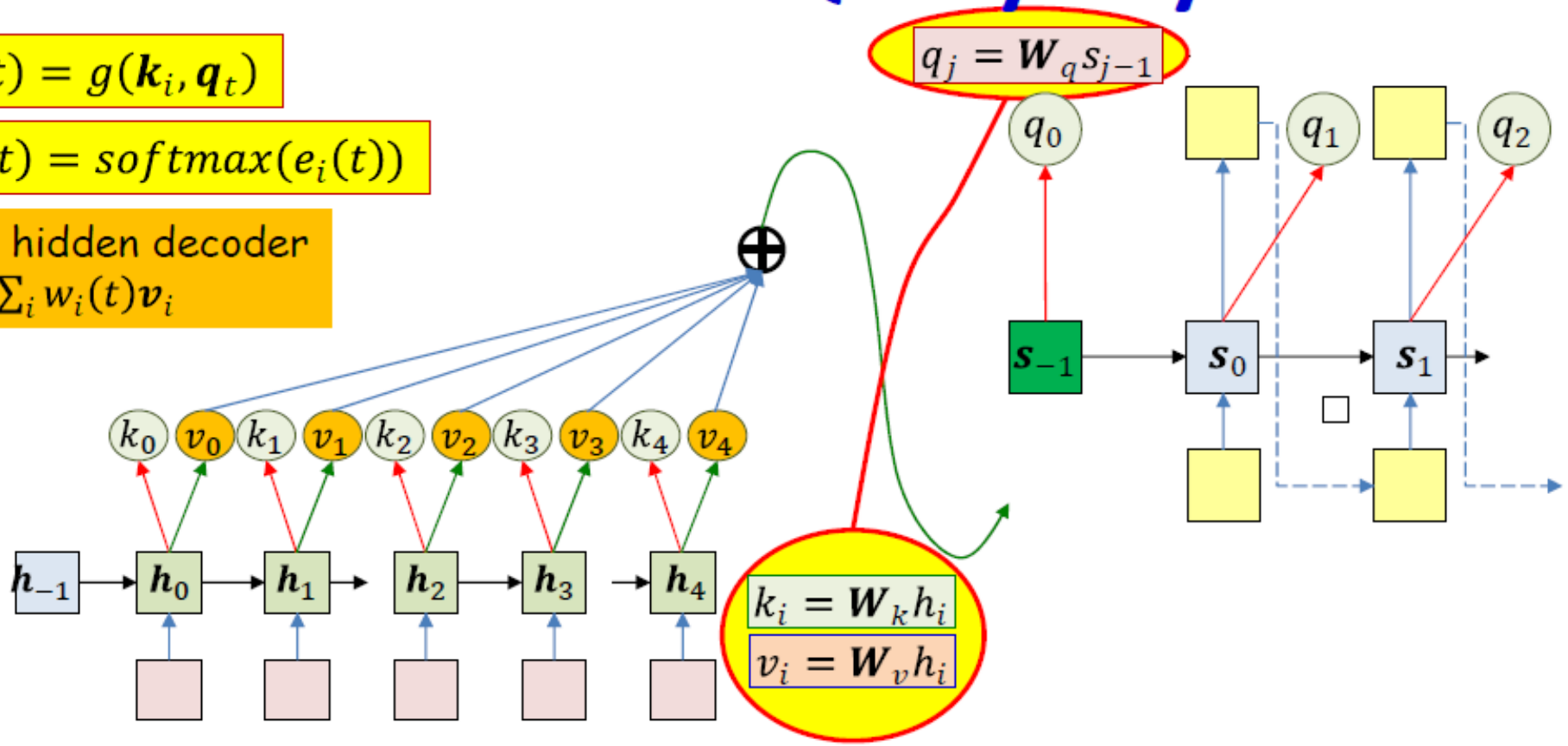
Attention mechanism

Modification: Query key value

$$e_i(t) = g(\boldsymbol{k}_i, \boldsymbol{q}_t)$$

$$w_i(t) = softmax(e_i(t))$$

Input to hidden decoder layer: $\sum_i w_i(t)\boldsymbol{v}_i$



RNN is a special case: $k_i = v_i = h_i$, $q_t = s_{t-1}$

Thank you!

SUSTech Southern University of Science and Technology